



ESCUELA DE INGENIERÍA DE FUENLABRADA

CIENCIA E INGENIERÍA DE DATOS

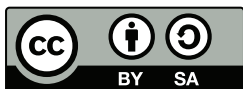
**TRABAJO FIN DE GRADO**

UN TÍTULO DE PROYECTO LARGO  
EN DOS LÍNEAS

Autor : Pablo Pardo Gutiérrez

Tutor : Dr. José Felipe Ortega Soto

Curso Académico 2025/2026



© 2026 Nombre y apellidos . Algunos derechos reservados.

El código fuente de este Trabajo Fin de Grado está disponible en:  
[enlacegithubaqui](#)

Este trabajo se entrega bajo la licencia Creative Commons  
**Reconocimiento-CompartirIgual** (by-sa). Para obtener  
la licencia completa, véase:  
<https://creativecommons.org/licenses/by-sa/4.0/deed.es>.

*Primera impresión, 18 de junio de 2026*

# **Trabajo Fin de Grado**

Título del Trabajo con Letras Capitales para Sustantivos y Adjetivos

**Autor :** Pablo Pardo Gutiérrez

**Tutor :** Dr. José Felipe Ortega Soto

La defensa del presente Proyecto Fin de Grado se realizó el día 3 de 2026, siendo calificada por el siguiente tribunal:

**Presidente:**

**Secretario:**

**Vocal:**

y habiendo obtenido la siguiente calificación:

**Calificación:**

Móstoles/Fuenlabrada, a de de 2026

*dedicacion  
aqui  
gracias*

# Agradecimientos

Aquí vienen los agradecimientos...

Hay más espacio para explayarse y explicar a quién agradeces su apoyo o ayuda para haber acabado el proyecto: familia, pareja, amigos, compañeros de clase...

También hay quien, en algunos casos, hasta agradecer a su tutor o tutores del proyecto la ayuda prestada...

# Resumen

Aquí viene un resumen del proyecto. Ha de constar de tres o cuatro párrafos, donde se presente de manera clara y concisa de qué va el proyecto. Han de quedar respondidas las siguientes preguntas:

- ¿De qué va este proyecto? ¿Cuál es su objetivo principal?
- ¿Cómo se ha realizado? ¿Qué tecnologías están involucradas?
- ¿En qué contexto se ha realizado el proyecto? ¿Es un proyecto dentro de un marco general?

Lo mejor es escribir el resumen al final.

# Summary

Here comes a translation of the “Resumen” into English. Please, double check it for correct grammar and spelling. As it is the translation of the “Resumen”, which is supposed to be written at the end, this as well should be filled out just before submitting.

# Índice general

|          |                                                                                        |           |
|----------|----------------------------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Introducción</b>                                                                    | <b>1</b>  |
| 1.0.1    | Motivación . . . . .                                                                   | 1         |
| 1.0.2    | Caso de Estudio: La ciudad de Melbourne . . . . .                                      | 2         |
| 1.1      | Estado del Arte . . . . .                                                              | 3         |
| 1.1.1    | Consolidación del IoT en la Gestión Inteligente del Aparcamiento . . . . .             | 3         |
| 1.1.2    | Evolución hacia el <i>Modern Data Stack</i> en Entornos IoT . . . . .                  | 4         |
| 1.1.3    | Modelado de Series Temporales y el Reto de los Datos Parcialmente Observados . . . . . | 4         |
| 1.1.4    | Evolución de la Analítica Predictiva . . . . .                                         | 5         |
| 1.2      | Objetivos del proyecto . . . . .                                                       | 7         |
| 1.2.1    | Objetivo general . . . . .                                                             | 7         |
| 1.2.2    | Objetivos específicos . . . . .                                                        | 7         |
| 1.3      | Planificación temporal . . . . .                                                       | 8         |
| 1.3.1    | Diagrama de Gantt . . . . .                                                            | 9         |
| 1.4      | Estructura de la memoria . . . . .                                                     | 9         |
| <b>2</b> | <b>Tecnologías</b>                                                                     | <b>11</b> |
| 2.1      | Lenguaje Base y Gestión de Entorno . . . . .                                           | 11        |
| 2.1.1    | Python                                                                                 | 11        |
| 2.1.2    | Gestor de dependencias: <i>uv</i> . . . . .                                            | 11        |
| 2.2      | Arquitectura de Ingesta y Almacenamiento . . . . .                                     | 12        |
| 2.2.1    | Apache Airflow . . . . .                                                               | 12        |
| 2.2.2    | DuckDB . . . . .                                                                       | 13        |
| 2.3      | Ciencia de Datos y <i>Deep Learning</i> . . . . .                                      | 13        |
| 2.3.1    | PyTorch . . . . .                                                                      | 13        |
| 2.3.2    | PyPOTS . . . . .                                                                       | 14        |
| 2.3.3    | TensorBoard . . . . .                                                                  | 15        |
| 2.4      | Visualización . . . . .                                                                | 15        |
| 2.4.1    | Grafana y Herramientas Geoespaciales . . . . .                                         | 15        |
| 2.4.2    | Quarto . . . . .                                                                       | 16        |



|          |                                                                    |           |
|----------|--------------------------------------------------------------------|-----------|
| <b>3</b> | <b>Diseño e implementación</b>                                     | <b>17</b> |
| 3.1      | Arquitectura general . . . . .                                     | 17        |
| 3.2      | Origen y Comprensión de los Datos . . . . .                        | 19        |
| 3.2.1    | Estructura del <i>dataset</i> histórico. Extracción automatizada   | 19        |
| 3.2.2    | Dataset geográfico . . . . .                                       | 20        |
| 3.3      | Análisis Exploratorio de Datos (EDA) . . . . .                     | 20        |
| 3.3.1    | Transformación y tipificado de variables . . . . .                 | 22        |
| 3.3.2    | Ingeniería de características ( <i>Feature Engineering</i> ) . . . | 22        |
| 3.3.3    | Análisis descriptivo y visualización . . . . .                     | 23        |
| 3.3.4    | Integración espacial y georreferenciación de los dispositivos      | 25        |
| 3.4      | Procesamiento de datos para PyPOTS . . . . .                       | 25        |
| 3.4.1    | Georreferenciación de observaciones . . . . .                      | 26        |
| 3.4.2    | Generación de <i>Grid</i> temporal . . . . .                       | 26        |
| 3.4.3    | Codificación cíclica de variables temporales . . . . .             | 27        |
| 3.4.4    | <i>Feature Selection</i> . . . . .                                 | 28        |
| 3.4.5    | Consolidación y exportación del conjunto de datos final            | 28        |
| 3.5      | Modelado de Series Temporales . . . . .                            | 29        |
| 3.5.1    | Fundamentos Teóricos de los Modelos de Imputación . .              | 29        |
| 3.5.2    | Fundamentos Teóricos de los Modelos de Predicción . .              | 31        |
| 3.5.3    | Modelos de Referencia. <i>Baselines</i> . . . . .                  | 33        |
| 3.6      | Implementación del Flujo de Entrenamiento y Predicción . . . .     | 35        |
| 3.6.1    | Preparación y Enmascaramiento de Datos . . . . .                   | 35        |
| 3.6.2    | Definición de Métricas de Evaluación . . . . .                     | 36        |
| 3.6.3    | Espacios de Búsqueda y Justificación Paramétrica . . .             | 38        |
| 3.6.4    | Orquestación y Automatización del Flujo . . . . .                  | 44        |
| 3.6.5    | Evaluación Final en <i>Test</i> . . . . .                          | 46        |
| 3.7      | Diseño del Pipeline ETL en tiempo real . . . . .                   | 47        |
| 3.7.1    | Lógica de consumo y esquema de almacenamiento de datos             | 47        |
| 3.7.2    | Orquestación del flujo . . . . .                                   | 49        |
| 3.8      | Despliegue en Producción . . . . .                                 | 49        |
| 3.8.1    | Arquitectura del sistema en Docker . . . . .                       | 50        |
| 3.8.2    | Lanzamiento y Monitorización . . . . .                             | 51        |
| 3.8.3    | Finalización de la ingesta y consolidación de datos . . .          | 51        |
| 3.9      | Transformación del conjunto de datos en tiempo real . . . . .      | 52        |
| 3.9.1    | Análisis estadístico de la dinámica temporal del sistema           | 53        |
| 3.9.2    | Procesamiento y reconstrucción de las señales . . . . .            | 53        |
| 3.10     | Clasificación espacial de zonas de estacionamiento . . . . .       | 54        |
| 3.10.1   | Extracción de puntos de interés (POIs) . . . . .                   | 55        |
| 3.10.2   | Algoritmo de clasificación ponderada . . . . .                     | 55        |
| 3.10.3   | Renderizado cartográfico e integración Web . . . . .               | 56        |

|          |                                                                         |           |
|----------|-------------------------------------------------------------------------|-----------|
| 3.11     | Desarrollo del portal de resultados con Quarto . . . . .                | 57        |
| 3.11.1   | Configuración . . . . .                                                 | 58        |
| 3.11.2   | Navegación . . . . .                                                    | 59        |
| 3.11.3   | Ejecución Dinámica y Representación Gráfica . . . . .                   | 59        |
| <b>4</b> | <b>Experimentación y resultados</b>                                     | <b>61</b> |
| 4.1      | Restricciones experimentales . . . . .                                  | 61        |
| 4.1.1    | Restricciones en la exploración paramétrica para CSDI . . . . .         | 61        |
| 4.1.2    | Ajuste de tamaño de ventana para MICN . . . . .                         | 62        |
| 4.2      | Análisis del Entrenamiento y Validación . . . . .                       | 63        |
| 4.2.1    | Análisis de convergencia en modelos de imputación . . . . .             | 63        |
| 4.2.2    | Análisis de convergencia en modelos de predicción . . . . .             | 64        |
| 4.3      | Resultados en Tareas de Imputación . . . . .                            | 65        |
| 4.4      | Resultados en Tareas de Predicción . . . . .                            | 67        |
| 4.5      | Evaluación Compuesta del Pipeline Predictivo . . . . .                  | 68        |
| 4.6      | Evaluación sobre el <i>dataset Out-Of-Time</i> . . . . .                | 69        |
| 4.6.1    | Resiliencia en la reconstrucción de la señal . . . . .                  | 70        |
| 4.6.2    | Degradación predecible y colapso arquitectónico (Forecasting) . . . . . | 70        |
| <b>5</b> | <b>Conclusiones y trabajos futuros</b>                                  | <b>72</b> |
| 5.1      | Consecución de objetivos . . . . .                                      | 72        |
| 5.2      | Aplicación de lo aprendido . . . . .                                    | 72        |
| 5.3      | Lecciones aprendidas . . . . .                                          | 73        |
| 5.4      | Trabajos futuros . . . . .                                              | 73        |
| 5.5      | Incorporación de código en la memoria . . . . .                         | 73        |
| 5.5.1    | Fuentes monoespaciadas . . . . .                                        | 74        |
| 5.6      | Contenido matemático y ecuaciones . . . . .                             | 75        |
| 5.6.1    | Ecuaciones en múltiples líneas y elementos alineados . . . . .          | 76        |
| 5.6.2    | Teoremas y conjuntos . . . . .                                          | 77        |
| <b>A</b> | <b>Contenido adicional</b>                                              | <b>78</b> |
| A.1      | Dimensiones de página . . . . .                                         | 78        |
|          | <b>Referencias</b>                                                      | <b>82</b> |

# Índice de figuras

|     |                                                                                                                                                                |    |
|-----|----------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 1.1 | Diagrama de Gantt ilustrando la temporalización de las fases del proyecto y sus respectivas tareas a lo largo de seis meses. . . . .                           | 10 |
| 3.1 | Arquitectura general del pipeline del sistema. . . . .                                                                                                         | 18 |
| 3.2 | Distribución de la variable objetivo ( <i>VehiclePresent</i> ). . . . .                                                                                        | 23 |
| 3.3 | Distribución del volumen de eventos de estacionamiento según la hora del día. . . . .                                                                          | 24 |
| 3.4 | Top de vías o zonas con mayor densidad de registros de aparcamiento ( <i>hotspots</i> ). . . . .                                                               | 24 |
| 3.5 | <i>High Performance Optimization</i> : flujo lógico de optimización de hiperparámetros. . . . .                                                                | 45 |
| 3.6 | Representación visual del flujo de tareas (DAG) en la interfaz de Apache Airflow. . . . .                                                                      | 49 |
| 3.7 | Monitorización de las ejecuciones recurrentes del <i>pipeline</i> ETL en el entorno de producción. . . . .                                                     | 51 |
| 3.8 | Distribución geográfica de las plazas de aparcamiento en Melbourne según su tipología urbana. . . . .                                                          | 57 |
| 3.9 | Detalle de la interfaz interactiva ( <i>tooltip</i> ) al inspeccionar un sensor individual. . . . .                                                            | 58 |
| 4.1 | Curvas de convergencia para el modelo CSDI. Izquierda: evolución de la función de pérdida en entrenamiento. Derecha: función de pérdida en validación. . . . . | 64 |
| 4.2 | Comparativa de la función de pérdida en entrenamiento entre MICN (naranja y rosado) y Transformer (azul y gris) para ambos horizontes predictivos. . . . .     | 65 |

# Índice de tablas

|     |                                                                                                                                                   |    |
|-----|---------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 3.1 | Diccionario de datos en DuckDB. . . . .                                                                                                           | 48 |
| 4.1 | MAE alcanzado en el conjunto de validación tras la búsqueda de hiperparámetros. . . . .                                                           | 63 |
| 4.2 | Métricas de evaluación en el conjunto de test para tareas de imputación. . . . .                                                                  | 66 |
| 4.3 | Métricas de evaluación en el conjunto de test para tareas de predicción a 24 y 48 horas. . . . .                                                  | 67 |
| 4.4 | Resultados del <i>pipeline</i> integrado. Comparativa de error MAE sobre datos originales frente a datos previamente imputados por SAITS. . . . . | 68 |
| 4.5 | Métricas de evaluación OOT para modelos de imputación. . . . .                                                                                    | 70 |
| 4.6 | Métricas de evaluación OOT para tareas de predicción a 24 y 48 horas. . . . .                                                                     | 71 |

# List of Listings

|     |                                                                                    |    |
|-----|------------------------------------------------------------------------------------|----|
| 3.1 | Espacio de búsqueda y configuración base para la arquitectura SAITS. . . . .       | 39 |
| 3.2 | Espacio de búsqueda y configuración base para la arquitectura CSDI. . . . .        | 40 |
| 3.3 | Espacio de búsqueda y configuración base para la arquitectura DLinear. . . . .     | 41 |
| 3.4 | Espacio de búsqueda y configuración base para la arquitectura Transformer. . . . . | 42 |
| 3.5 | Espacio de búsqueda y configuración base para la arquitectura MICN. . . . .        | 43 |
| 5.1 | Lectura de un fichero *.csv y tipado de datos. . . . .                             | 74 |

# 1 Introducción

El despliegue de redes IoT (Internet de las Cosas) representa la piedra angular en la transición de las urbes tradicionales hacia las *Smart Cities* (o Ciudades Inteligentes). Históricamente, la planificación urbana y la gestión del tráfico se basaban en estudios estadísticos puntuales que quedaban obsoletos rápidamente. Sin embargo, la integración del IoT ha transformado la infraestructura física de las ciudades en un ecosistema digital dinámico e interconectado.

En este nuevo paradigma, los sensores IoT actúan como el “sistema nervioso” de la ciudad. Estos dispositivos, integrados en el asfalto, semáforos, farolas o contenedores, son capaces de capturar, procesar y transmitir parámetros físicos en tiempo real. La verdadera disrupción tecnológica del IoT urbano no reside únicamente en la recolección aislada de datos, sino en la capacidad de generar un flujo continuo de información a gran escala (*Big Data*). Esto permite a las administraciones públicas y a los sistemas automatizados pasar de una gobernanza reactiva, que actúa cuando el problema ya se ha manifestado, a una proactiva y predictiva, lo que permite maximizar la eficiencia y anticipar casuísticas de interés.

## 1.0.1 Motivación

El impacto del IoT en la movilidad urbana es especialmente crítico. En las últimas décadas, el crecimiento demográfico y la rápida urbanización han transformado la congestión vehicular en uno de los mayores desafíos para las administraciones públicas, generando un impacto negativo tanto en el medio ambiente, por el aumento de emisiones de gases de efecto invernadero, como en la calidad de vida de los ciudadanos. Diversos estudios urbanísticos señalan que el tráfico de agitación —aquel provocado por vehículos que circulan a baja velocidad buscando activamente aparcamiento— es responsable de hasta un 30% de la congestión en los centros urbanos, lo que representa una fuente masiva de contaminación atmosférica y acústica [15].

Para mitigar este problema, la instalación de sensores electromagnéticos o de infrarrojos en las plazas de aparcamiento en superficie permite monitorizar la ocupación exacta en tiempo real. No obstante, la implementación práctica de estas redes de sensores se enfrenta a un desafío técnico ineludible: la volatilidad y la pérdida de integridad de los datos. Al tratarse de dispositivos físicos distribuidos en un entorno hostil (la vía pública), los sensores están sujetos a fallos de conectividad, latencia de red, averías por factores climáticos o agotamiento de baterías [6]. Consecuentemente, las series temporales de datos quedan discontinuas y plagadas de valores faltantes, lo que distorsiona severamente cualquier análisis visual o intento de toma de decisiones informada.

Por consiguiente, la motivación principal de este proyecto surge de la necesidad de construir un ecosistema integral y resiliente que aborde el ciclo de vida completo del dato urbano. No es suficiente con instalar sensores; es imperativo diseñar una arquitectura de ingeniería de datos robusta y automatizada que ingeste y almacene esta información en tiempo real. Asimismo, es crucial la aplicación de técnicas vanguardistas de *Deep Learning* capaces de comprender los patrones espaciotemporales subyacentes para imputar y reconstruir la información faltante cuando el hardware falla. Solo garantizando esta calidad e integridad técnica, el flujo incesante de datos del IoT puede ser explotado eficazmente, transformando los datos crudos en valor real y democratizado para la ciudadanía.

### 1.0.2 Caso de Estudio: La ciudad de Melbourne

Para materializar y validar la arquitectura propuesta, este proyecto toma como escenario de aplicación la red de aparcamientos en superficie de la ciudad de Melbourne (Australia). La elección de esta urbe responde a criterios técnicos y de disponibilidad de datos que la convierten en un entorno de pruebas óptimo para el desarrollo del proyecto.

En primer lugar, el gobierno local de Melbourne es pionero a nivel mundial en políticas de *Open Data*, al ofrecer acceso público y documentado a su infraestructura IoT. A diferencia de otras grandes ciudades que únicamente proporcionan datos agregados de aparcamientos subterráneos, Melbourne ofrece una granularidad a nivel de plaza individual.

En segundo lugar, la plataforma proporciona una dualidad de acceso crítica para este proyecto. Por un lado, un repositorio histórico masivo, como el *dataset* del año 2019 escogido para este proyecto, el cual es indispensable para la fase de entrenamiento de modelos de *Deep Learning*. Por otro lado, una API REST que

emite el estado de los sensores en tiempo real, lo cual habilita la implementación de un *pipeline* de orquestación continua.

Finalmente, al tratarse de una red de sensores físicos expuesta a condiciones urbanas reales durante años, el conjunto de datos de Melbourne no es un entorno estéril o simulado. Contiene intrínsecamente las imperfecciones, latencias, fallos de batería y valores faltantes descritos en la motivación de este trabajo. Esta naturaleza imperfecta es precisamente lo que legitima y hace necesaria la aplicación de herramientas de imputación avanzadas. A su vez, una vez restaurada la integridad temporal de la red, este entorno real es idóneo para desplegar y evaluar modelos predictivos (*forecasting*) de vanguardia, de forma que se completa del ciclo completo, desde el dato crudo hasta la inteligencia predictiva.

## 1.1 Estado del Arte

### 1.1.1 Consolidación del IoT en la Gestión Inteligente del Aparcamiento

En la última década, el Internet de las Cosas (IoT) ha superado su fase experimental para consolidarse como la infraestructura subyacente de las *Smart Cities*. Este avance ha sido impulsado por el despliegue de redes de bajo consumo y largo alcance (LPWAN) y la transición hacia el *Edge Computing*, lo cual hace posible sensorizar masivamente infraestructuras críticas urbanas [1].

Dentro de la movilidad urbana, el *Smart Parking* destaca como uno de los campos con mayor madurez tecnológica. La información generada por los sensores ha dejado de ser puramente descriptiva para integrarse en tres grandes casos de uso operativos: (I) sistemas de guiado en tiempo real para reducir el tráfico de agitación; (II) tarificación dinámica (*Dynamic Pricing*) basada en la demanda espacial; y (III) control telemático de infracciones (*Enforcement*). Sin embargo, el verdadero potencial de estos sistemas requiere trascender la monitorización actual hacia la analítica predictiva, lo cual exige infraestructuras de procesamiento de datos altamente eficientes [9].



### 1.1.2 Evolución hacia el *Modern Data Stack* en Entornos IoT

El procesamiento del ingente volumen de datos espaciotemporales generado por las redes ha evidenciado las limitaciones de las arquitecturas monolíticas tradicionales ETL (*Extract, Transform, Load*). Históricamente, la gestión de estos flujos requería bases de datos pesadas y costosos clústeres computacionales.

Actualmente, el estado del arte a nivel industrial y de investigación converge hacia el paradigma del *Modern Data Stack* (MDS). Esta evolución se caracteriza por la adopción de herramientas ágiles, desacopladas y orientadas al código (*Data-as-Code*). En la fase de orquestación, planificadores dinámicos como Apache Airflow se han estandarizado para gestionar dependencias complejas de ingesta. Simultáneamente, el paradigma de almacenamiento analítico ha experimentado una disrupción con la aparición de motores OLAP (procesamiento analítico en línea) en proceso (*in-process*), entre los que DuckDB destaca como su máximo exponente actual. Estos sistemas permiten ejecutar consultas vectorizadas sobre datos IoT directamente en entornos locales o *edge*, superando la latencia inherente a las bases de datos cliente-servidor tradicionales [13].

### 1.1.3 Modelado de Series Temporales y el Reto de los Datos Parcialmente Observados

Pese a contar con infraestructuras de datos robustas, la extracción de valor predictivo se topa con el desafío técnico expuesto en la motivación: la naturaleza intermitente de la señal física. Los algoritmos clásicos de *Machine Learning* y las redes neuronales estándar tienen un comportamiento errático ante la presencia de huecos o valores nulos en las secuencias temporales.

Actualmente, la vanguardia investigadora no se centra únicamente en la predicción pura, sino en el modelado de Datos Temporales Parcialmente Observados (*Partially Observed Time Series* o POTS).

Para abordar esta problemática, la comunidad científica ha transitado desde heurísticas simples hacia *frameworks* de aprendizaje profundo especializados en imputación. Librerías recientes de código abierto, como PyPOTS [4], logran unificar la evaluación de distintos enfoques. En el contexto de este proyecto, la reconstrucción de la señal se aborda evaluando empíricamente la evolución de este campo mediante tres grandes enfoques:

- **Heurísticas Clásicas (Media y LOCF).** Tradicionalmente, la imputación en entornos industriales dependía de sustituciones estadísticas simples como la imputación por la Media Global o el acarreo del último valor observado (LOCF, *Last Observation Carried Forward*). Aunque computacionalmente triviales, estas técnicas asumen una distribución estática que ignora la dinámica espaciotemporal real del tráfico urbano. En la actualidad se emplean únicamente como (*baselines*) comparativos.
- **Imputación basada en Auto-Atención (SAITS).** El paradigma moderno de reconstrucción ha sido dominado por arquitecturas como SAITS (*Self-Attention-based Imputation for Time Series*) [5]. Este enfoque utiliza un mecanismo dual de atención conjunta que captura tanto la correlación temporal (histórica) como la espacial (entre distintos sensores) simultáneamente. De esta forma logra inferir datos faltantes masivos con alta precisión algorítmica.
- **Modelos Generativos Probabilísticos (CSDI).** Las últimas investigaciones en imputación adoptan los modelos de difusión estocástica, popularizados en la generación de imágenes. Arquitecturas como CSDI (*Conditional Score-based Diffusion Models for Probabilistic Time Series Imputation*) [17] no solo predicen un valor determinista para el sensor caído, sino que generan una distribución probabilística del tráfico: se logra cuantificar la incertidumbre, lo cual es altamente valioso en entornos reales.

### 1.1.4 Evolución de la Analítica Predictiva

Una vez garantizada la integridad de las series temporales mediante técnicas de imputación, el siguiente desafío reside en la anticipación de la demanda mediante el modelado predictivo. Históricamente, la predicción de tráfico y ocupación urbana ha estado dominada por modelos estadísticos paramétricos, como ARIMA, y posteriormente por Redes Neuronales Recurrentes (RNNs) como las LSTM (*Long Short-Term Memory*). Sin embargo, estos enfoques presentan limitaciones severas al intentar modelar secuencias temporales excesivamente largas o capturar estacionalidades concurrentes [19].

El punto de inflexión en el análisis secuencial se produjo con la irrupción de la arquitectura Transformer [19]. Originalmente diseñados para el procesamiento del lenguaje natural, los Transformers introdujeron el mecanismo de auto-atención (*Self-Attention*), lo que permite a los modelos correlacionar cualquier punto del pasado con el futuro de forma directa, sin sufrir la degradación de memoria característica de las redes recurrentes. En consecuencia, se produjo

una proliferación de modelos basados en atención orientados a series temporales.

No obstante, el modelado temporal contemporáneo ha comenzado a cuestionar el monopolio de los Transformers debido a su alta complejidad computacional (cuadrática respecto a la longitud de entrada) y su tendencia al sobreajuste frente a ruidos estacionales. Como respuesta, el Estado del Arte actual se ha diversificado en tres grandes paradigmas competitivos que este proyecto busca evaluar empíricamente:

- **El paradigma de Atención (Transformer).** Representa el enfoque de aprendizaje profundo puro, que emplea matrices de atención multicabeza (*Multi-Head Attention*) para descubrir correlaciones globales no lineales complejas a largo plazo.
- **El paradigma Convolutivo (MICN).** Busca mitigar el coste de la atención temporal con arquitecturas recientes como MICN (*Multi-scale Isometric Convolutional Network*) [20], las cuales han demostrado que el uso de convoluciones isométricas unidimensionales a múltiples escalas permite extraer características locales y globales de la serie temporal de manera mucho más eficiente y ligera.
- **El paradigma de Descomposición Lineal (DLinear).** De forma contraintuitiva frente a la tendencia hacia arquitecturas cada vez más profundas, investigaciones recientes han demostrado que un conjunto de capas lineales simples puede superar a los Transformers más avanzados [22]. DLinear descompone la serie temporal en un componente de tendencia (mediante medias móviles) y un componente estacional, aplicando mapeos lineales directos que resultan extremadamente rápidos y robustos frente a anomalías.

La convergencia de estos tres paradigmas proporciona un marco de evaluación idóneo para determinar la relación coste-beneficio computacional a la hora de predecir la movilidad urbana en entornos reales.

## 1.2 Objetivos del proyecto

### 1.2.1 Objetivo general

Desarrollar un sistema integral de ingeniería de datos e Inteligencia Artificial capaz de predecir la disponibilidad de plazas de aparcamiento en superficie. La consecución de este objetivo exige el diseño de una arquitectura robusta que ingiera telemetría IoT en tiempo real y aborde el ciclo de vida completo del dato: desde la captura de señales físicas, a las que se aplica una reconstrucción algorítmica de la serie temporal mediante modelos de imputación, hasta la aplicación de arquitecturas predictivas del estado del arte, para anticipar la movilidad a corto y medio plazo. Este enfoque busca transformar datos brutos en inteligencia urbana.

### 1.2.2 Objetivos específicos

Para alcanzar el objetivo general, el proyecto se desglosa en las siguientes metas técnicas secuenciales:

- **Realizar un Análisis Exploratorio de Datos (EDA) sobre registros históricos.** Analizar masivamente el histórico de la red de sensores para identificar distribuciones, extraer estacionalidades complejas de la movilidad urbana y detectar anomalías en los tiempos de estacionamiento que condicionen el modelado posterior.
- **Desplegar una infraestructura de ingesta y almacenamiento continuo en tiempo real.** Diseñar y orquestar un *pipeline* automatizado mediante Apache Airflow para consumir telemetría viva de la API de la ciudad de Melbourne durante un periodo ininterrumpido de un mes. El objetivo es someter al sistema a condiciones reales de producción y almacenar el flujo resultante en una base de datos analítica centralizada (DuckDB).
- **Reconstruir la señal temporal mediante modelos de imputación SOTA.** Entrenar y evaluar modelos neuronales avanzados en el conjunto de datos histórico para imputar los valores faltantes (*missing data*) en el *dataset* real recopilado. Esta meta es crítica para subsanar los fallos inherentes de red y hardware de la sensórica IoT y poder garantizar la integridad de los datos antes de la fase predictiva.

- **Predecir la ocupación futura mediante modelado predictivo.** Diseñar un experimento comparativo utilizando arquitecturas de vanguardia para series temporales: modelos de descomposición lineal (DLinear), mecanismos de auto-atención pura (Transformer) y redes convolucionales isométricas (MICN). El objetivo es predecir la disponibilidad de plazas a horizontes de 24 y 48 horas bajo las mismas restricciones computacionales y de contexto.
- **Desarrollar un cuadro de mando analítico en tiempo real.** Construir un *dashboard* interactivo conectando Grafana directamente al motor OLAP de DuckDB, permitiendo la monitorización fluida del estado de la red de aparcamientos y las métricas de rendimiento.
- **Construir un visualizador geoespacial contextualizado.** Representar la topología de la red de sensores en un mapa 2D interactivo, categorizando las plazas de aparcamiento según su proximidad y relación con distintos Puntos de Interés (POIs) urbanos.
- **Divulgar los resultados mediante un entorno web interactivo.** Desarrollar y desplegar un *portfolio* digital utilizando Quarto, estructurado para exponer la metodología científica, la arquitectura de datos implementada y las conclusiones algorítmicas del proyecto de manera accesible y reproducible.

### 1.3 Planificación temporal

Para garantizar la consecución de los objetivos descritos y asegurar la viabilidad técnica, el ciclo de vida del proyecto se ha estructurado en cuatro fases de desarrollo iterativo e incremental.

- **Fase 1: Conceptualización y despliegue de infraestructura.** En esta etapa inicial se llevó a cabo la revisión bibliográfica del estado del arte y el diseño teórico de la arquitectura. Seguidamente, se configuraron los entornos virtualizados y se implementaron los flujos de orquestación con Apache Airflow, y se estableció la conexión temprana con la API de Melbourne para posteriormente iniciar la ventana de ingesta y almacenamiento continuo en DuckDB.
- **Fase 2: Procesamiento histórico y *Feature Engineering*.** Esta fase se centró en la adecuación del conjunto de datos masivo correspondiente al año 2019. Se ejecutaron labores de limpieza, integración de metadatos geoespaciales y, fundamentalmente, el remuestreo (*resampling*) de los eventos

vehiculares asíncronos. Esto permitió estandarizar la información y construir las matrices temporales con valores nulos explícitos requeridas para el modelado.

- **Fase 3: Modelado predictivo y visualizaciones.** Con los datos estructurados, se procedió al diseño, entrenamiento y ajuste de los modelos de *Deep Learning* de PyPOTS. En paralelo, se construyó el ecosistema de visualización conectando Grafana a la base de datos analítica y desarrollando mapas interactivos para la representación espacial de las plazas y su relación con lugares próximos relevantes.
- **Fase 4: Validación y Cierre.** La etapa final consistió en la validación del ecosistema, aplicando el modelo entrenado sobre los datos recopilados en tiempo real para ejecutar la imputación de valores faltantes. Finalmente, se desplegó el entorno web divulgativo mediante Quarto y se procedió a la extracción de conclusiones y revisión exhaustiva de la presente memoria académica.

El proyecto se ha desarrollado durante el periodo comprendido entre los meses de enero y junio. El avance técnico se ha abordado mediante una metodología de trabajo continua, integrando la ejecución práctica de las fases descritas con la redacción y revisión progresiva de la presente memoria académica.

### 1.3.1 Diagrama de Gantt

Para ilustrar visualmente la planificación temporal detallada en la sección anterior, se ha elaborado el diagrama de Gantt correspondiente al ciclo de vida del proyecto. Esta representación gráfica permite observar la duración y secuenciación de las tareas específicas que componen cada una de las cuatro fases principales.

## 1.4 Estructura de la memoria

El resto del presente documento se estructura en cuatro capítulos adicionales, organizados de manera secuencial para reflejar el ciclo de vida del proyecto:

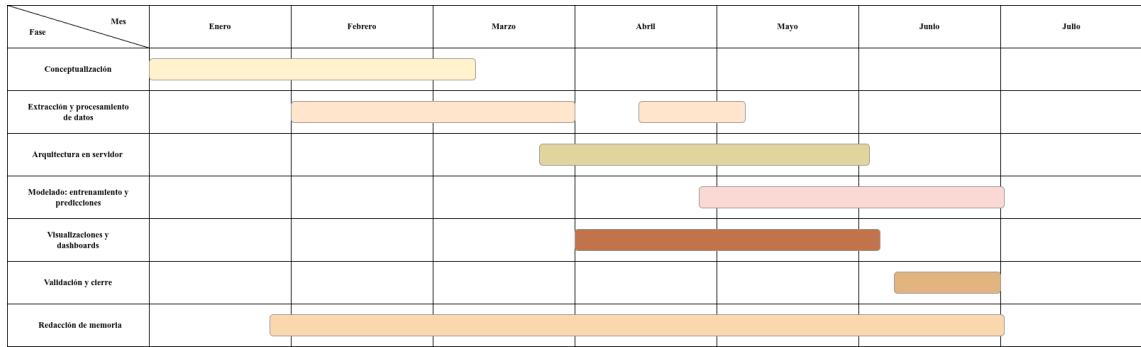


Figura 1.1: Diagrama de Gantt ilustrando la temporalización de las fases del proyecto y sus respectivas tareas a lo largo de seis meses.

- El **capítulo 2** (Tecnologías empleadas) detalla las herramientas, lenguajes y *frameworks* utilizados para la consecución del proyecto. Se justifica la elección de cada pieza de *software* dentro del paradigma del *Modern Data Stack*.
- El **capítulo 3** (Diseño e Implementación) constituye el núcleo técnico de la memoria. En él se describe exhaustivamente el desarrollo del *pipeline* de datos, los procesos de limpieza y *resampling* del conjunto histórico, así como el flujo entrenamiento de los modelos de imputación y predicción. Asimismo, se detallan el diseño arquitectónico de la orquestación en tiempo real y la construcción del ecosistema de visualización geoespacial.
- El **capítulo ??** (Experimentación y Resultados) presenta los hallazgos obtenidos tras la evaluación de los modelos en el conjunto de datos histórico y en el de datos en tiempo real, así como información relevante sobre su entrenamiento. Se realiza una comparación y análisis de los diferentes resultados en diferentes ventanas de contexto y configuraciones.
- Finalmente, el **capítulo 5** (Conclusiones) sintetiza las deducciones finales extraídas tras la finalización del trabajo. Posteriormente, se evalúa el grado de cumplimiento de los objetivos planteados y se proponen futuras líneas de investigación y desarrollo que podrían derivarse de esta base.

## 2 Tecnologías

Para dar respuesta a las necesidades del proyecto, se ha diseñado una arquitectura de datos robusta, escalable y reproducible. La selección de herramientas se ha fundamentado en los estándares actuales de la industria y el paradigma del *Modern Data Stack*. En este capítulo se detallan las tecnologías empleadas, así como la justificación técnica que motiva su adopción frente a otras alternativas del mercado.

### 2.1 Lenguaje Base y Gestión de Entorno

#### 2.1.1 Python

Python se ha consolidado como el lenguaje de programación de referencia en el ecosistema de la Ciencia de Datos y la Inteligencia Artificial, gracias a su sintaxis legible y su vasta comunidad de desarrollo [18]. En este proyecto, Python actúa como el hilo conductor que vertebra todas las fases de la arquitectura, desde los *scripts* de extracción de datos hasta el entrenamiento de las redes neuronales.

#### 2.1.2 Gestor de dependencias: *uv*

Una práctica ineludible en cualquier proyecto de ingeniería de *software* es garantizar la reproducibilidad del entorno virtual. Para la gestión de dependencias y paquetes, se ha adoptado *uv*, una herramienta de vanguardia desarrollada en el lenguaje Rust que ha irrumpido recientemente como sustituto de alto rendimiento para el gestor tradicional *pip*.

La justificación técnica para la adopción de *uv* radica en su eficiencia arquitectónica extrema. Al estar compilado en Rust, aprovecha los estándares modernos de empaquetado de Python (como PEP 518 y PEP 621), ya que lee las dependencias directamente desde archivos de configuración `.toml` sin necesidad



de ejecutar código de terceros, un cuello de botella histórico en *pip*. Además, *uv* rechaza paquetes mal formados y aplica reglas estrictas que ignoran formatos obsoletos, lo que se traduce en una mayor seguridad y estabilidad del entorno.

A nivel de rendimiento, *uv* descarga múltiples paquetes de manera concurrente en lugar de realizar peticiones secuenciales. Destaca especialmente su implementación de un sistema de caché global mediante enlaces duros (*hard links*); si un paquete ya ha sido instalado previamente en otro proyecto de la máquina, *uv* enlaza los archivos en lugar de duplicarlos. Esta característica, sumada al uso del algoritmo de resolución de dependencias *PubGrub* para evitar conflictos de versiones, reduce drásticamente los tiempos de instalación y el consumo de almacenamiento en disco.

## 2.2 Arquitectura de Ingesta y Almacenamiento

### 2.2.1 Apache Airflow

Apache Airflow es una plataforma de código abierto diseñada para crear, planificar y monitorizar flujos de trabajo programáticos (*workflows*). A diferencia de herramientas de planificación clásicas basadas en el sistema operativo (como *cron*), Airflow permite definir las tuberías de datos como código (*Data-as-Code*) mediante Grafos Acíclicos Dirigidos (DAGs), lo que proporciona un control granular sobre las dependencias entre tareas.

En la arquitectura de este proyecto, Airflow actúa como el orquestador principal encargado de automatizar las extracciones recurrentes de la API del Ayuntamiento de Melbourne. Se ha desplegado en un servidor virtualizado, configurado con los requisitos de seguridad y aislamiento de red pertinentes. Esta infraestructura garantiza una ejecución ininterrumpida.

Su elección frente a otros orquestadores se fundamenta en su resiliencia. Airflow gestiona de forma nativa los reintentos ante caídas o fallos de conexión (*retries*), alerta sobre cuellos de botella y proporciona una interfaz web robusta para monitorizar el estado del *pipeline* en tiempo real, lo cual es crítico para una arquitectura de *micro-batching*.

## 2.2.2 DuckDB

DuckDB es un sistema de gestión de bases de datos relacionales en proceso (*in-process*), diseñado específicamente para cargas de trabajo de procesamiento analítico en línea (OLAP). Frente a motores tradicionales embebidos como SQLite, que almacenan la información fila por fila (ideales para transacciones puntuales), DuckDB emplea una arquitectura de almacenamiento orientada a columnas. Además, DuckDB se ejecuta dentro de la aplicación, lo cual implica que no requiere un servidor externo, ni gestión de clústeres, ni una configuración de infraestructura compleja [13].

Esta aproximación columnar, combinada con un motor de ejecución de consultas vectorizado, permite que las operaciones analíticas complejas y las agregaciones temporales masivas se resuelvan a una velocidad significativamente superior. El motor procesa bloques de datos (vectores) en una sola instrucción de la CPU, lo que minimiza la sobrecarga computacional.

Se ha integrado DuckDB en el proyecto por su sinergia absoluta con el ecosistema de Python y, muy especialmente, por su capacidad de ejecutar sentencias SQL estándar directamente sobre *DataFrames* de Pandas. Al operar en el mismo espacio de memoria que el proceso que lo invoca, DuckDB permite realizar consultas relacionales analíticas complejas (incluyendo sentencias **JOIN**, agrupaciones o funciones de ventana) al consultar directamente variables nativas de Python, sin necesidad de copiar, exportar o transferir los datos previamente a un servidor externo. Esta versatilidad híbrida, que aúna la expresividad del lenguaje SQL con la flexibilidad analítica de Pandas, lo convierte en la base de datos ideal para transformar y almacenar el flujo incesante de datos espaciotemporales procedentes de Airflow.

## 2.3 Ciencia de Datos y *Deep Learning*

### 2.3.1 PyTorch

PyTorch es un *framework* de aprendizaje automático de código abierto desarrollado por el laboratorio de investigación de Inteligencia Artificial de Meta (FAIR). Se ha establecido como el estándar en la investigación académica y la industria para el desarrollo de arquitecturas de *Deep Learning*.

La adopción de PyTorch como el motor computacional subyacente de este proyecto se justifica por dos características arquitectónicas fundamentales. En primer

lugar, su paradigma de grafos computacionales dinámicos (*Dynamic Computation Graphs*), que permite modificar la estructura de la red neuronal durante el tiempo de ejecución, facilitando enormemente la depuración del código. En segundo lugar, su integración nativa con la plataforma CUDA de NVIDIA, lo que permite paralelizar el entrenamiento de los modelos delegando las multiplicaciones matriciales masivas (esenciales en arquitecturas como el Transformer) a la unidad de procesamiento gráfico (GPU).

### 2.3.2 PyPOTS

PyPOTS (*A Python Toolbox for Data Mining on Partially Observed Time Series*) es una librería especializada de código abierto construida sobre el ecosistema de PyTorch. Fue impulsada originalmente por el investigador Wenjie Du y es respaldada por una comunidad académica internacional con miembros de instituciones como la Universidad de Oxford y el King's College London [4]. Está diseñada específicamente para el modelado, clasificación y predicción de series temporales que presentan valores nulos o discontinuidades estructurales. Su arquitectura unifica implementaciones de vanguardia del estado del arte algorítmico, proporcionando una API estandarizada de alto nivel.

En este trabajo, el papel de PyPOTS es bimodal, dando respuesta a las dos fases críticas del tratamiento de la serie temporal:

- **Fase de Imputación.** Permite entrenar modelos de *Deep Learning* capaces de inferir el estado de los sensores físicos estropeados. A diferencia de las técnicas de interpolación matemática clásica que solo observan datos históricos aislados, PyPOTS analiza el contexto global de la red, extrayendo representaciones latentes profundas del comportamiento espacial y temporal urbano para reconstruir la señal con extrema precisión.
- **Fase de Predicción.** Una vez restaurada la integridad del conjunto de datos, PyPOTS provee las arquitecturas predictivas modernas necesarias para proyectar la ocupación futura. La librería integra nativamente un amplio abanico de familias algorítmicas con arquitecturas simples y otras muy complejas. Es importante señalar que algunas de las arquitecturas complejas han tenido que ser descartadas para este proyecto debido a su inasumible coste computacional y desbordamiento de parámetros.

Adicionalmente, cabe destacar que PyPOTS incluye integración nativa con NNI (*Neural Network Intelligence*), el *framework* de código abierto de Microsoft diseñado para la optimización automatizada de hiperparámetros (HPO). Aunque

se tiene plena constancia de su potencial analítico para explorar el espacio de búsqueda, en el presente trabajo no se ha podido explotar esta tecnología en su totalidad debido a restricciones estrictas de *hardware*. Los intentos de orquestar algoritmos avanzados de búsqueda saturaban la capacidad física del entorno local, colapsando los núcleos de la CPU al 100 % de su capacidad y agotando por completo la memoria VRAM del dispositivo CUDA. Por consiguiente, la optimización paramétrica se ha adaptado a estos techos computacionales para garantizar la viabilidad del entrenamiento, como se puede observar en la sección ??.

### 2.3.3 TensorBoard

TensorBoard constituye una plataforma de visualización interactiva diseñada originalmente para el ecosistema TensorFlow, aunque en la actualidad ofrece integración nativa con PyTorch y, por extensión, con la librería PyPOTS. Esta herramienta permite monitorizar e inspeccionar el flujo computacional y las métricas de evaluación durante el entrenamiento de las diversas arquitecturas.

La justificación técnica para su adopción radica en la necesidad de analizar la evolución de la función de pérdida durante el entrenamiento, así como las métricas de error en validación (como el MSE) a lo largo de las épocas.

TensorBoard proyecta estas magnitudes matemáticas en gráficas interactivas, lo cual resulta indispensable para diagnosticar problemas de convergencia y detectar *overfitting*. Adicionalmente, al almacenar los registros de distintas ejecuciones, la plataforma permite superponer y comparar múltiples configuraciones de hiperparámetros (HPO) en un mismo plano espacial, lo cual permite también verificar el correcto accionamiento de las reglas de parada temprana.

## 2.4 Visualización

### 2.4.1 Grafana y Herramientas Geoespaciales

Para la monitorización visual de la información procesada, se ha optado por Grafana, una plataforma de código abierto líder en observabilidad y *Business Intelligence* (BI). Grafana destaca por su capacidad para conectarse a múltiples

fuentes de datos de forma nativa. En este proyecto, se alimenta directamente del motor analítico DuckDB, lo que permite generar cuadros de mando (*dashboards*) interactivos que reflejan métricas de interés sobre los datos de ocupación.

En el plano del análisis espacial, se han incorporado librerías de mapeo interactivas para Python, como Folium. Esta herramienta actúa como puente tecnológico hacia la biblioteca de JavaScript Leaflet, lo cual permite renderizar mapas de calor y representar geográficamente la red de sensores. Su uso es clave para analizar la disponibilidad de plazas en función de su proximidad a Puntos de Interés (POIs) específicos, como centros comerciales o áreas de ocio.

### 2.4.2 Quarto

Para democratizar el acceso a los resultados y la metodología del proyecto, se ha empleado Quarto. Se trata de un sistema de publicación científica y técnica de código abierto, construido sobre la herramienta conversora universal Pandoc. Quarto permite entrelazar texto narrativo en formato *Markdown* con código ejecutable de Python, para generar documentos dinámicos de alta calidad estética [2].

En este contexto, Quarto se ha utilizado para el desarrollo y despliegue del entorno web estructurado del proyecto. Esta tecnología facilita la compilación de un *portfolio* interactivo que agrupa la arquitectura de datos, las visualizaciones generadas por Grafana y Folium, y las conclusiones extraídas de los modelos predictivos, lo que ofrece una experiencia divulgativa alternativa a la tradicional memoria estática en PDF.

## 3 Diseño e implementación

El presente capítulo constituye el núcleo técnico de este trabajo. En él se describe de manera exhaustiva el proceso de desarrollo de la plataforma, detallando las decisiones de diseño, el tratamiento de la información y la construcción del *pipeline* predictivo. El objetivo es proporcionar una visión clara y reproducible de cómo se han integrado las diferentes tecnologías para dar solución al problema del pronóstico de aparcamiento mediante *Deep Learning*.

### 3.1 Arquitectura general

Para garantizar la escalabilidad y robustez del sistema, se ha diseñado una arquitectura orientada a los datos, dividida en fases secuenciales que abarcan desde la ingesta hasta la visualización final. La estructura global del proyecto se ilustra en la Figura 3.1.

Tal y como se observa en la Figura 3.1, el sistema se articula de forma secuencial, dividiendo sus primeras fases (Capas 1 y 2) en dos flujos de trabajo diferenciados según la naturaleza temporal de los datos:

- **Flujo histórico.** Ejecutado de forma local mediante *scripts* de Python, se encarga de la extracción de datos desde el portal de *Open Data* de Melbourne y posteriores transformaciones. Estos datos serán empleados en entrenar el modelo de PyPOTS.
- **Flujo en tiempo real.** Orquestado íntegramente por Apache Airflow, interactúa de forma periódica con la API del Ayuntamiento para obtener los datos en tiempo real, estandarizarlos y mantener el sistema actualizado mediante el almacenamiento en DuckDB. Estos datos serán destinados para realizar la inferencia una vez el modelo ha sido entrenado.
- **Motor de modelado predictivo.** Realiza el entrenamiento con los datos históricos y ejecutar la inferencia para generar las predicciones futuras.

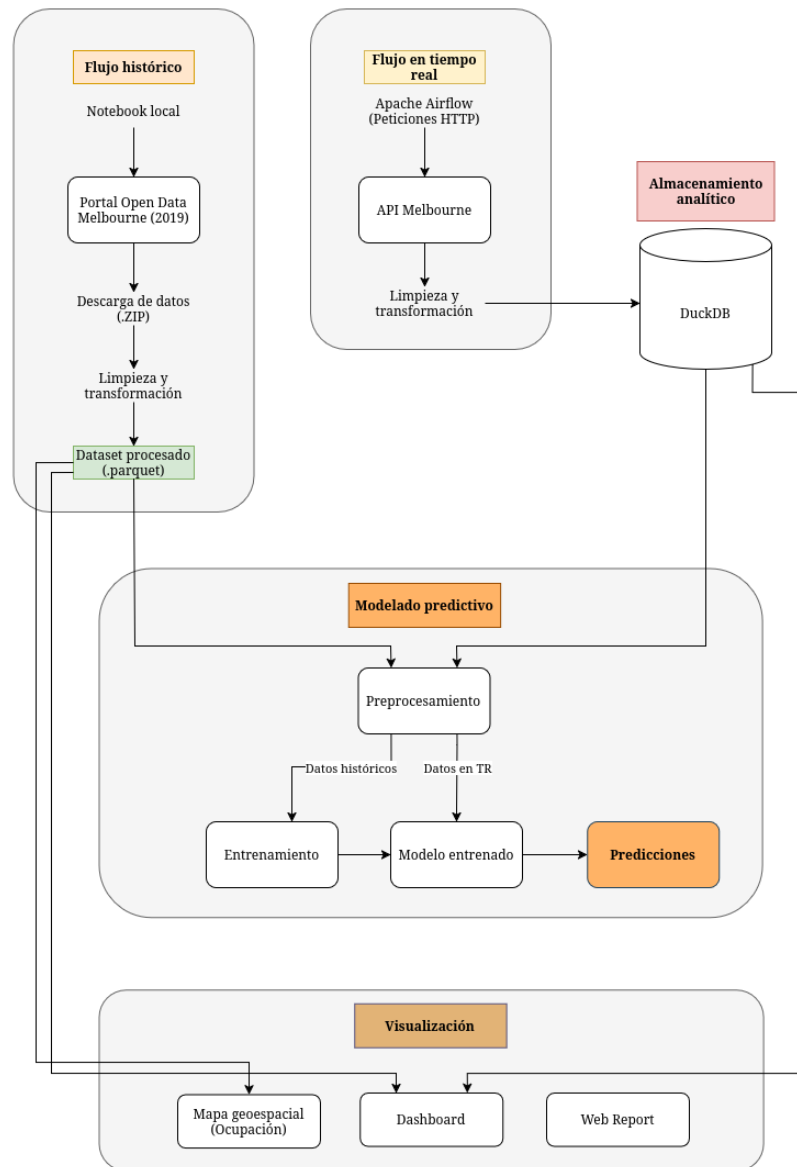


Figura 3.1: Arquitectura general del pipeline del sistema.

- **Interfaz de consumo y visualización.** Integración de herramientas específicas según el caso de uso: Grafana para la monitorización de métricas y predicciones, Folium para la representación interactiva de la ocupación geográfica, y Quarto para la web del proyecto.

## 3.2 Origen y Comprensión de los Datos

El primer paso crítico en cualquier proyecto de analítica predictiva es la adquisición y comprensión del dominio de los datos. Para este trabajo, se ha tomado como base empírica el portal de *Open Data* del Ayuntamiento de Melbourne.

### 3.2.1 Estructura del *dataset* histórico. Extracción automatizada

Para la fase de entrenamiento y experimentación inicial, se recurre al *dataset* histórico estático correspondiente al año 2019. La elección de este año en particular responde a una limitación operativa y contextual importante: debido a las anomalías de movilidad y a los cambios en las políticas de retención de datos durante la pandemia de la COVID-19, los conjuntos de datos posteriores a 2020 presentan vacíos de información o no fueron almacenados de forma persistente. Por tanto, 2019 representa el último ciclo temporal completo y fidedigno de la movilidad urbana ordinaria (incluyendo estacionalidad anual, festividades locales y variaciones estacionales).

Para garantizar la reproducibilidad del entorno de desarrollo y evitar el manejo manual de archivos masivos, la obtención de este conjunto de datos se ha procedimentado mediante un *pipeline* de extracción automatizado en Python. Haciendo uso de librerías nativas como *requests* para peticiones HTTP, se orquestó la conexión al repositorio original para la descarga del archivo comprimido por bloques (*chunks*) para evitar colapsar la memoria RAM.

Una vez finalizada la transferencia, el sistema localiza el archivo (*.zip*), extrae su contenido programáticamente utilizando la librería *zipfile* y almacena los datos tabulares en formato CSV para su posterior procesamiento.

Este conjunto masivo de datos, compuesto por más de 42 millones de registros anuales, presenta un paradigma de almacenamiento orientado a eventos. Cada



fila del documento no representa el estado general o agregado de una calle en un intervalo de tiempo dado, sino un evento asíncrono e individual: la llegada (*ArrivalTime*) o la salida (*DepartureTime*) física de un vehículo en un sensor o plaza de aparcamiento específica (*DeviceId* o *Bay ID*), acompañado de su marca temporal exacta.

Esta alta granularidad, si bien es excelente para capturar la realidad del tráfico de agitación plaza por plaza, requiere de un procesamiento intensivo para transformar un registro de eventos asíncronos en una serie temporal regular y estructurada con la que posteriormente poder modelar.

#### 3.2.2 Dataset geográfico

De forma análoga a la obtención del histórico de ocupación, el *pipeline* de extracción se reutiliza para descargar un segundo conjunto de datos: el inventario geográfico de plazas de aparcamiento. Este archivo contiene las coordenadas geográficas precisas (*latitud* y *longitud*) y la descripción del segmento de vía (*road segment description*) de la infraestructura de la ciudad. Su extracción automatizada sigue los mismos principios de eficiencia y reproducibilidad que para el dataset histórico.

### 3.3 Análisis Exploratorio de Datos (EDA)

Antes de aplicar cualquier técnica de modelado o de diseñar el esquema de la base de datos analítica, es imperativo realizar un análisis exploratorio de datos. Esta fase persigue un doble objetivo: comprender las distribuciones subyacentes de los datos de movilidad en Melbourne, así como las variables de interés, e identificar anomalías en las mediciones físicas de los sensores.

Adicionalmente, se ha identificado que el *dataset* principal, pese a su exhaustividad temporal, carece de componentes geográficas explícitas más allá de identificadores alfanuméricos de las calles. Por tanto, esta fase exploratoria también contempla el enriquecimiento de los datos mediante el cruce con el inventario geográfico de la ciudad, un paso esencial para dotar al análisis de contexto topográfico.

Debido al gran volumen de información, las fases iniciales de diseño del algoritmo de limpieza se han prototipado sobre una muestra representativa de 500.000

filas, un tamaño significativo para obtener conclusiones generalizables al conjunto completo de datos. Esta estrategia permitió agilizar las iteraciones de desarrollo y optimizar el consumo de memoria RAM antes de escalar el proceso al clúster completo.

Tras visualizar las observaciones del *dataset* en formato *DataFrame* de *Pandas*, se realiza una interpretación a alto nivel de las variables presentes, clasificándolas en cuatro categorías principales:

- **Variables temporales:**

- *ArrivalTime* (**str**): fecha y hora exacta en la que el sensor detectó la llegada del vehículo.
- *DepartureTime* (**str**): fecha y hora exacta en la que el vehículo abandonó la plaza.
- *DurationMinutes* (**str**): tiempo total de estacionamiento (diferencia temporal entre la llegada y la salida).

- **Variables de estado:**

- *VehiclePresent* (**bool**): variable objetivo (*target*). Indica la presencia física de un vehículo sobre el sensor durante ese periodo.
- *InViolation* (**bool**): indica si el vehículo excedió el tiempo permitido, incurriendo en una infracción.
- *Sign* (**str**) / *SignPlateID* (**float64**): texto y código identificador de la señal física de tráfico de la calle. Estas variables presentan una alta proporción de valores nulos.

- **Identificadores de plazas:**

- *DeviceId* (**int64**): número de serie electrónico del sensor.
- *BayId* (**int64**): identificador de la plaza de aparcamiento.
- *StreetMarker* (**str**): código alfanumérico identificativo. La tupla formada por (*DeviceId*, *StreetMarker*) constituye una clave única.

- **Variables de ubicación:**

- *AreaName* (**str**): barrio o distrito general al que pertenece la plaza.
- *StreetName* (**str**) / *StreetId* (**int64**): nombre y código numérico de la calle principal.

- *BetweenStreet1* (**str**) / *BetweenStreet1ID* (**int64**): primera calle transversal que acota el tramo de estacionamiento.
- *BetweenStreet2* (**str**) / *BetweenStreet2ID* (**int64**): segunda calle transversal que acota el tramo.
- *SideName* (**str**) / *SideOfStreetCode* (**str**) / *SideOfStreet* (**int64**): identificadores de la acera de estacionamiento y su ubicación geográfica (Norte, Sur, Este, Oeste). Existe una equivalencia predefinida entre códigos (1-C, 2-E, 3-N, 4-S, 5-W).

#### 3.3.1 Transformación y tipificado de variables

Los datos brutos presentan inconsistencias de formato que requieren una estandarización previa. En primer lugar, la variable de duración (*DurationMinutes*) fue convertida de cadena de texto a valor numérico de coma flotante (**float**). Asimismo, las marcas de tiempo (*ArrivalTime* y *DepartureTime*), almacenadas originalmente como texto plano, se transforman a objetos *datetime* nativos, lo cual es indispensable para habilitar la correcta indexación temporal y el posterior remuestreo de la serie.

#### 3.3.2 Ingeniería de características (*Feature Engineering*)

Con las variables temporales correctamente tipificadas, se procede a la descomposición de las fechas para extraer componentes estacionales subyacentes. A partir de la fecha de llegada, se generaron tres nuevas características fundamentales:

- **Hora del día (*Hour*)**: busca capturar los picos de tráfico laboral diario.
- **Día de la semana (*WeekDay*)**: busca entender el comportamiento de la movilidad entre días laborables y fines de semana.
- **Mes del año (*Month*)**: ayuda a modelar la macro-estacionalidad, como periodos vacacionales o cambios de estación.

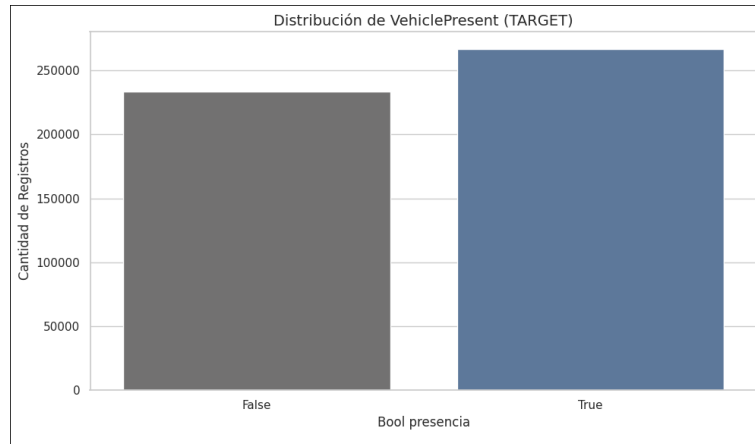


Figura 3.2: Distribución de la variable objetivo (*VehiclePresent*).

#### 3.3.3 Análisis descriptivo y visualización

Una vez estandarizado el conjunto de datos y generadas las nuevas variables, se continua con un análisis visual descriptivo para obtener *insights* sobre el comportamiento de la movilidad.

En primer lugar, se analiza la distribución de la variable objetivo binaria (*VehiclePresent*), la cual indica la ocupación física de las plazas. Este paso es crítico para evaluar el balanceo de clases antes de la fase de modelado.

Como se aprecia en la Figura 3.2, se evalúa la proporción de registros correspondientes a plazas ocupadas frente a plazas libres. En problemas de *Deep Learning*, un desbalanceo severo en la variable a predecir podría inducir a la red neuronal a apostar sistemáticamente por la clase mayoritaria, reduciendo drásticamente su capacidad de generalización. La visualización de esta distribución permite confirmar que no hay un desbalanceo notable, pues ambas clases presentan una representación equilibrada.

Posteriormente, se analiza el volumen de registros de aparcamiento en función de la hora del día, con el fin de identificar los patrones de rotación diarios.

Tal y como se ilustra en la Figura 3.3, la movilidad urbana en las plazas sensorizadas presenta un comportamiento fuertemente cíclico y dependiente del horario laboral. Se observa un valle profundo de inactividad durante la madrugada, seguido de un incremento pronunciado a partir de las 08:00 horas. El volumen de eventos se mantiene en niveles máximos durante las horas centrales del día, reflejando la



Figura 3.3: Distribución del volumen de eventos de estacionamiento según la hora del día.

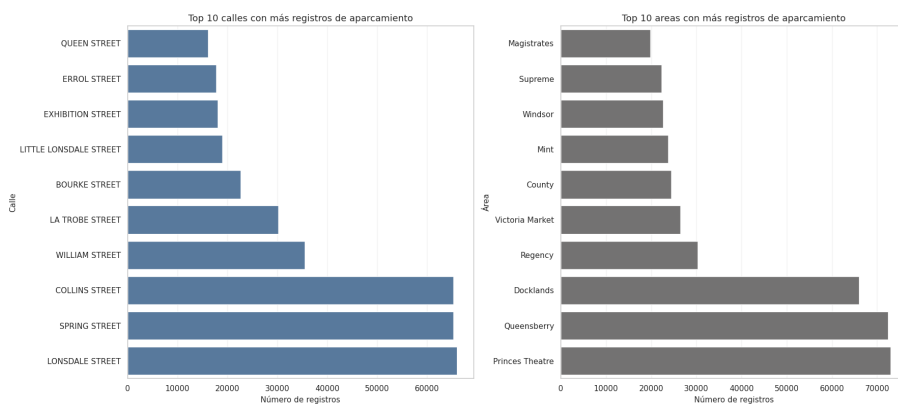


Figura 3.4: Top de vías o zonas con mayor densidad de registros de aparcamiento (*hotspots*).

alta rotación comercial y laboral, para finalmente descender de forma paulatina a partir de las 18:00 horas aproximadamente.

Por último, se evalúa la distribución espacial de estos eventos para identificar los principales puntos de congestión o *hotspots* de la ciudad. El análisis geográfico representado en la Figura 3.4 revela que la demanda de aparcamiento no se distribuye de manera uniforme, sino que se concentra asimétricamente en unas pocas vías principales. Estas calles, que acumulan la mayor densidad de registros, pueden corresponder a arterias de algún distrito central de negocios o a zonas con alta concentración de servicios. Esta información es importante, pues estas zonas serán áreas prioritarias para el modelo predictivo. En relación a las áreas más concurridas destaca *Docklands*, seguida por *Queensberry* y el entorno del *Princes Theatre*.

### 3.3.4 Integración espacial y georreferenciación de los dispositivos

Para dotar al sistema de contexto geoespacial y poder enriquecer la información de entrada de los modelos predictivos, resulta indispensable conocer la ubicación física exacta de cada plaza de aparcamiento. Sin embargo, el conjunto de datos histórico carece nativamente de las coordenadas geográficas de los sensores. Para solventar esta necesidad, se descarga un conjunto de datos con coordenadas geográficas de los sensores (longitud y latitud), proveniente del ayuntamiento de Melbourne y se realiza un *merge* de datos entre el conocido *dataset* histórico transaccional de 2019 y este nuevo conjunto de datos espaciales.

El proceso de georreferenciación se ejecuta de forma secuencial. En primer lugar, se construye el subconjunto de sensores únicos del histórico a partir de las variables identificativas del dispositivo y su contexto vial (**BayId**, **StreetName**...). A continuación, dado que los identificadores entre ambas fuentes no están estandarizados, se aplica una función de emparejamiento basada en la normalización textual de la toponimia (calle principal y calles transversales), comparando dichas descripciones con los segmentos del conjunto espacial. Cuando existe coincidencia, se asigna al sensor el par de coordenadas de latitud y longitud del segmento correspondiente. Finalmente, se realiza un filtrado para eliminar los sensores no georreferenciados y, posteriormente, se eliminan duplicados de coordenadas para conservar ubicaciones únicas.

El resultado de esta integración es un *dataset* de sensores geolocalizados único. Este subconjunto sirve de base de conocimiento espacial para la clasificación de plazas de parking en función de los puntos de interés desarrollada más adelante (véase la sección 3.10), así como para alimentar con contexto geográfico a las distintas arquitecturas de *deep learning*.

## 3.4 Procesamiento de datos para PyPOTS

Esta sección detalla las transformaciones finales a los datos para la obtención del *dataset* finalmente utilizado como entrada para los modelos. Este se obtiene al realizar la integración del *dataset* histórico procesado tras el EDA (con las nuevas variables temporales generadas) con el *dataset* espacial, también procesado previamente, que contiene los ids sin duplicados de los sensores y su longitud y latitud asociada.

### 3.4.1 Georreferenciación de observaciones

El proceso de georreferenciación se ejecuta de forma secuencial. En primer lugar, se construye un diccionario a partir del dataframe espacial, donde cada clave es un identificador de dispositivo (**DeviceId**) y el valor asociado es un diccionario con las coordenadas 'lat' y 'lon'. Se asignan las coordenadas a cada registro del dataset principal mediante la función `map`, que busca el identificador de dispositivo en el diccionario y extrae la latitud y longitud correspondientes, de forma que se eliminan aquellos sensores cuyo identificador de dispositivo no se encuentra en el dataframe espacial: el total de observaciones se reduce a 260 303.

Este nuevo dataset con información enriquecida debe ser adecuado a los requisitos estructurales y matemáticos de la librería PyPOTS como se detalla en las siguientes subsecciones.

### 3.4.2 Generación de *Grid* temporal

El conjunto de datos transaccional original registra los eventos de forma asíncrona; es decir, documenta las llegadas y salidas de los vehículos en instantes de tiempo arbitrarios y variables. Sin embargo, los modelos predictivos de series temporales requieren secuencias de observación regulares, donde la distancia cronológica entre dos fotogramas consecutivos sea estrictamente constante. Para resolver esta discrepancia estructural, se ha implementado un proceso de remuestreo temporal basado en cuadrícula.

En primer lugar, se define un vector temporal continuo con una frecuencia fija de 15 minutos, abarcando desde el primer registro hasta el final de la ventana de estudio. A continuación, se realiza el producto cartesiano entre este vector temporal y los identificadores únicos de los sensores (**DeviceId**). Esto crea un esqueleto tabular homogéneo donde cada plaza de aparcamiento posee una fila exacta para cada intervalo de 15 minutos, independientemente de si en ese momento concreto realmente ocurrió un evento o no.

Para poblar este esqueleto con el estado real de la calle, se aplica una técnica de cruce por proximidad temporal basada en el último estado conocido (**merge\_asof**). Este algoritmo evalúa cada marca de tiempo de la cuadrícula (por ejemplo, las 10:15 h) y retrocede cronológicamente hasta encontrar el último evento registrado para ese sensor, propagando su estado de ocupación (**VehiclePresent**) hacia adelante. Finalmente, se inputan nulos en la secuencia en función de la siguiente restricción: si la marca de tiempo evaluada en la

cuadrícula es estrictamente posterior a la hora de salida (**DepartureTime**) del último vehículo detectado, la ocupación de la plaza pasa a registrarse como nula.

Esta generación intencionada de valores nulos es un paso de diseño crítico, ya que materializa explícitamente los “huecos temporales” a imputar en la serie por los algoritmos de imputación de PyPOTS.

### 3.4.3 Codificación cíclica de variables temporales

Las variables temporales extraídas en la fase de análisis exploratorio, tales como la hora, el día de la semana y el mes, poseen una naturaleza inherentemente cíclica. Si estas características se introducen en la red neuronal como valores enteros secuenciales, el modelo percibirá una distancia matemática irreal; por ejemplo, interpretaría una separación máxima entre las 23:00 y las 00:00, cuando en la realidad cronológica representan instantes consecutivos. Si bien técnicas clásicas de codificación posicional como el *One-Hot Encoding* logran mitigar este sesgo aritmético, se generan vectores ortogonales independientes que destruyen por completo la secuencialidad del tiempo: por ejemplo, se sumiría que un lunes es matemáticamente tan distinto de un martes como de un domingo.

Para preservar la continuidad y el ciclo natural del tiempo sin comprometer la interpretabilidad del modelo, se ha implementado una técnica de codificación cíclica mediante transformaciones trigonométricas. Cada variable temporal se ha descompuesto en dos componentes espaciales continuas proyectadas sobre un círculo unitario. Para cualquier instante temporal  $x$ , las transformaciones aplicadas se definen como:

$$x_{sin} = \sin\left(\frac{2\pi \cdot x}{\max(x)}\right)$$
$$x_{cos} = \cos\left(\frac{2\pi \cdot x}{\max(x)}\right)$$

Gracias a esta proyección, instantes fronterizos como las 23:59 y las 00:01 poseen coordenadas trigonométricas prácticamente idénticas. Esto permite a la red neuronal interpretar correctamente la transición fluida entre ciclos diarios, semanales y mensuales, optimizando el aprendizaje de patrones estacionales sin incrementar desproporcionadamente la dimensionalidad horizontal del conjunto de datos.



### 3.4.4 Feature Selection

Como paso final, se realiza la fase de selección de características. El objetivo primordial de esta etapa consiste en depurar el volumen de información para retener únicamente aquellos atributos que aporten un valor predictivo directo, unívoco y generalizable, y su vez eliminar el ruido y la redundancia que puedan inducir al sobreentrenamiento o penalizar la eficiencia computacional de las redes neuronales.

Bajo este criterio metodológico, se descartan todas las características de naturaleza textual o administrativa previamente analizados en la fase exploratoria, tales como los nombres descriptivos de las calles, las descripciones de las señales de tráfico o los indicadores de infracción, dado que carecen de propiedades aritméticas explotables por el modelo. Asimismo, identificadores secundarios como los códigos de plaza o de acera se omiten al estar implícitamente representados por el número de serie electrónico del sensor.

### 3.4.5 Consolidación y exportación del conjunto de datos final

Como culminación del flujo de preprocesamiento específico, el conjunto de datos definitivo se exporta en *Parquet*. El *dataset* resultante se caracteriza por ser una matriz de 4 239 840 filas y 11 columnas.

El vector de variables que compone formalmente el esquema de este conjunto de datos final está constituido de forma estricta por los siguientes componentes:

- **Identificador del sensor (`DeviceId`):** clave única que identifica cada dispositivo físico en la red.
- **Marca de tiempo indexada (`Timestamp`):** registro temporal homogeneizado e indexado en intervalos regulares de 15 minutos.
- **Indicador de ocupación (`VehiclePresent`):** variable objetivo transformada a *float* para su procesamiento en la red.
- **Coordenadas espaciales de longitud (`lon`) y latitud (`lat`):** componentes geográficas que proporcionan el posicionamiento del dispositivo.
- **Transformaciones trigonométricas de la hora (`Hour_sin` y `Hour_cos`):** par de variables continuas que modelan el ciclo diario.

- **Transformaciones trigonométricas del día (WeekDay\_sin y WeekDay\_cos):** par de variables continuas que modelan el ciclo semanal.
- **Transformaciones trigonométricas del mes (Month\_sin y Month\_cos):** par de variables continuas que modelan el ciclo anual.

## 3.5 Modelado de Series Temporales

El presente capítulo sienta las bases teóricas y matemáticas de las arquitecturas de aprendizaje profundo (*Deep Learning*) seleccionadas para acometer las tareas de imputación y predicción a futuro. A lo largo de las siguientes subsecciones, se desgana la topología interna de modelos que representan el actual estado del arte en la literatura científica. El estudio inicia con las técnicas de imputación, para posteriormente adentrarse en las arquitecturas predictivas.

### 3.5.1 Fundamentos Teóricos de los Modelos de Imputación

Para abordar el desafío matemático de la reconstrucción de series temporales con valores ausentes, el presente trabajo selecciona dos arquitecturas neuronales que representan el estado del arte en sus respectivas familias algorítmicas. En primer lugar, se prueba una aproximación determinista basada en mecanismos de atención (SAITS). Posteriormente, se aplica un enfoque probabilístico fundamentado en modelos de difusión estocástica (CSDI). Ambos algoritmos han demostrado su superioridad teórica y empírica frente a métodos estadísticos tradicionales al ser evaluados en *benchmarks* públicos de gran exigencia, tales como el conjunto de datos médicos PhysioNet-2012 [16] y el registro meteorológico Beijing PM2.5 [8].

#### Arquitectura SAITS

El modelo SAITS (*Self-Attention-based Imputation for Time Series*), propuesto por Du, Cote y Liu [5], adapta la exitosa arquitectura *Transformer* [19] a las particularidades topológicas de las series temporales con observaciones parciales. A diferencia de las redes recurrentes (RNN) clásicas, propensas al desvanecimiento del gradiente en secuencias largas, SAITS emplea un mecanismo de *self-attention*

capaz de capturar dependencias temporales complejas de forma simultánea a lo largo de toda la ventana de observación.

El núcleo matemático de esta arquitectura se fundamenta en un diseño de optimización conjunta mediante bloques de atención con enmascaramiento diagonal. Para procesar la entrada, el sistema proyecta la serie temporal en tres famosas matrices fundamentales: *queries* ( $Q$ ), *keys* ( $K$ ) y *values* ( $V$ ). La relación de importancia entre los distintos instantes temporales se calcula mediante la función de atención multiplicativa escalada:

$$\text{Attention}(Q, K, V) = \text{softmax} \left( \frac{QK^T}{\sqrt{d_k}} \right) V \quad (3.1)$$

Donde  $d_k$  representa la dimensión de las claves. Para prevenir el *data leakage*, SAITS introduce una matriz de enmascaramiento asimétrico que impide a un paso temporal observar su propio valor durante el proceso de autoatención, forzando a la red a estimar dicho valor a partir del contexto circundante. Para prevenir el *data leakage*, SAITS introduce una matriz de enmascaramiento asimétrico (DMSA, *Diagonally-Masked Self-Attention*) que impide a un paso temporal observar su propio valor durante el proceso de autoatención, forzando a la red a estimar dicho valor a partir del contexto circundante.

A nivel arquitectónico, el modelo no emplea un único bloque, sino una estructura dual en cascada. El primer bloque extrae las representaciones latentes preliminares, mientras que el segundo bloque refina la imputación. A nivel de salida, SAITS conforma un modelo determinista que se entrena mediante una estrategia de optimización conjunta (*joint optimization*). La función de pérdida combina dos objetivos: la Tarea de Reconstrucción Observada (ORT), que asegura la fidelidad frente a los datos reales, y la Tarea de Imputación Enmascarada (MIT), que minimiza el error absoluto (MAE) sobre los valores nulos artificialmente inyectados durante el entrenamiento. Esta combinación garantiza que el tensor finalmente emitido mantenga tanto la coherencia local como la dinámica global de la serie.

### Arquitectura CSDI

Por su parte, el algoritmo CSDI (*Conditional Score-based Diffusion Models for Probabilistic Time Series Imputation*), introducido por Tashiro et al. [17], aborda el problema de la imputación desde una perspectiva probabilística y generativa. Al basarse en los Modelos Probabilísticos de Difusión de Eliminación de Ruido

(DDPM) [7], esta arquitectura no busca predecir un valor único, sino modelar la distribución de probabilidad completa de los datos ausentes condicionada a las observaciones existentes.

El funcionamiento interno de CSDI se divide en dos procesos estocásticos de tipo cadena de Markov. El primer paso, denominado proceso de difusión hacia adelante (*forward process*), consiste en inyectar ruido gaussiano de forma iterativa y gradual sobre los datos originales durante  $T$  pasos continuos, hasta transformar la serie temporal en ruido blanco puro. La distribución de transición para un paso temporal  $t$  se define como:

$$q(x_t|x_{t-1}) = \mathcal{N}(x_t; \sqrt{1 - \beta_t}x_{t-1}, \beta_t\mathbf{I}) \quad (3.2)$$

Donde  $\beta_t$  controla la varianza del ruido inyectado. El segundo paso, conocido como proceso inverso (*reverse process*), entrena a una red neuronal paramétrica para aproximar la *score function* y revertir el ruido paso a paso. En el contexto específico de CSDI, este proceso inverso se encuentra fuertemente condicionado por los valores observados y por la inyección de *embeddings* que codifican el instante de tiempo y la característica multivariante. Para procesar dicha información, la red paramétrica interna implementa un mecanismo de atención bidimensional: una capa de atención temporal que modela las dependencias cronológicas, seguida de una capa de atención de características (*feature attention*) que captura las correlaciones espaciales entre las distintas variables del sistema (ej. relaciones entre diferentes sensores o variables meteorológicas).

Al concluir la inferencia, el modelo probabilístico no devuelve un único tensor, sino una distribución de posibles trayectorias válidas para la serie temporal. Para someter este resultado a una métrica de precisión geométrica como el MAE o el MSE, el sistema debe extraer un valor puntual representativo al calcular la media o la mediana sobre el conjunto de trayectorias generadas, lo que dota a la predicción final de una gran robustez frente a anomalías locales.

### 3.5.2 Fundamentos Teóricos de los Modelos de Predicción

A diferencia de la imputación, que busca estimar valores ausentes mediante el contexto global, la predicción consiste en la generación de secuencias futuras a partir de una ventana de observación histórica. Para ello, se han seleccionado tres arquitecturas representativas de los paradigmas actuales de *Deep Learning*:

la descomposición lineal, el mecanismo de atención pura y la convolución isométrica.

### Arquitectura DLinear

El modelo DLinear, propuesto por Zeng et al. [22], desafía la hegemonía de las arquitecturas complejas basadas en *Transformers* mediante un enfoque de descomposición de series temporales. Su diseño se fundamenta en la premisa de que gran parte de la señal temporal puede explicarse mediante dos componentes básicos: una tendencia a largo plazo y una estacionalidad cíclica.

DLinear descompone la serie de entrada  $X$  en dos flujos:

$$X_{\text{trend}} = \text{MovingAverage}(X), \quad X_{\text{seasonal}} = X - X_{\text{trend}} \quad (3.3)$$

Donde la tendencia ( $X_{\text{trend}}$ ) se extrae mediante un filtro de media móvil unidimensional. Posteriormente, cada componente es procesado de manera independiente mediante redes neuronales de una sola capa lineal (*fully connected*). A diferencia de los modelos basados en atención, que son invariantes a la permutación y requieren codificaciones posicionales artificiales, las capas lineales de DLinear mapean directamente la serie histórica completa hacia la ventana de predicción. Esta simplicidad no solo reduce drásticamente el coste computacional y el riesgo de sobreajuste (*overfitting*), sino que preserva intrínsecamente el orden semántico del tiempo, lo que permite al modelo extrapolar tendencias a largo plazo con un número mínimo de parámetros. La salida final se obtiene mediante la suma de las proyecciones de ambos componentes.

### Arquitectura Transformer

La arquitectura *Transformer*, extrapolada del procesamiento de lenguaje natural [19], ha sido adaptada al *forecasting* mediante un esquema *encoder-decoder*. En este dominio, el *encoder* procesa la serie temporal de entrada para generar representaciones latentes, mientras que el *decoder* genera la secuencia futura.

El componente central es el mecanismo de *self-attention*, que permite a cada paso temporal "atender" a cualquier otro punto de la serie, capturando dependencias de largo alcance que las redes recurrentes ignoran. Matemáticamente, el modelo mapea la secuencia de entrada  $S_{\text{input}}$  hacia un espacio de alta dimensión, con

múltiples proyecciones en paralelo (*multi-head attention*) para capturar diversas dinámicas temporales (e.g ciclos diarios y semanales). Sin embargo, a pesar de su indiscutible éxito en el dominio discreto del procesamiento de lenguaje, la aplicación del Transformer estándar a series temporales continuas presenta retos significativos. Por un lado, su complejidad temporal cuadrática  $O(L^2)$  respecto a la longitud de la secuencia de entrada ( $L$ ) restringe drásticamente la capacidad de procesar ventanas históricas extensas. Por otro lado, la atención basada en puntos (*point-wise attention*) es altamente susceptible a anomalías y ruido sensorio de alta frecuencia, lo que a menudo dificulta la extracción de patrones globales estables en el análisis del tráfico urbano a largo plazo.

#### Arquitectura MICN

El modelo MICN (*Multi-scale Isometric Convolutional Network*), introducido por Wang et al. [20], propone una alternativa eficiente a los *Transformers* mediante la implementación de convoluciones isométricas. Esta arquitectura reconoce que las series temporales poseen patrones tanto locales como globales y que tratar ambos niveles con un mismo filtro es subóptimo.

A nivel estructural, MICN integra un diseño jerárquico basado en el submuestreo a múltiples escalas. En cada bloque de la red, la señal se bifurca para ser procesada simultáneamente por dos mecanismos de convolución unidimensional. El primer mecanismo extrae características locales granulares (cambios a corto plazo), mientras que el segundo mecanismo, basado en convoluciones isométricas, amplía dinámicamente el campo receptivo para modelar el contexto global (tendencias a largo plazo). La característica “isométrica” asegura que la red mantenga la alineación posicional de la señal, mitigando la pérdida de resolución espacial inherente al *pooling* estándar. Al fusionar la visión local y global mediante capas de retroalimentación cruzada, MICN logra capturar correlaciones temporales complejas con una complejidad computacional lineal  $O(L)$ , erigiéndose como una solución más ligera y robusta que la autoatención pura para horizontes predictivos cortos y medios.

#### 3.5.3 Modelos de Referencia. *Baselines*

Para garantizar la validez científica de los resultados, resulta imprescindible establecer un marco de referencia empírico. Se incorporan *baselines* que permitan establecer una cota inferior de rendimiento aceptable para cada tarea.

#### Baselines de Imputación: Inercia y Tendencia Central

En el contexto específico de imputación de series temporales, el método determinista de referencia por excelencia es la propagación de la última observación válida (LOCF, por sus siglas en inglés, *Last Observation Carried Forward*) [10]. Esta heurística asume que el estado dinámico del sistema tiende a permanecer estático en el corto plazo, hipótesis que cobra especial relevancia técnica en el dominio de los sensores de estacionamiento, donde la ocupación de una plaza suele mantenerse constante durante intervalos temporales.

Junto al algoritmo LOCF, se desarrollan métodos de imputación por medidas de tendencia central, concretamente la Media Global. Mientras que LOCF establece la cota de rendimiento basada en la inercia temporal, las aproximaciones estadísticas unidimensionales rellenan los valores ausentes al calcular el promedio histórico de la variable, al ignorar por completo la secuencia cronológica [12].

#### Baseline de Predicción: El límite de la linealidad (DLinear)

En el ámbito de la predicción futura, la selección del modelo de referencia responde a una necesidad metodológica propia de la investigación contemporánea en Inteligencia Artificial. Durante los últimos años, la proliferación de arquitecturas tipo *Transformers* ha eclipsado a los métodos paramétricos tradicionales bajo la premisa de que las relaciones no lineales profundas son necesarias para predecir el futuro. Sin embargo, estudios recientes [22] han cuestionado severamente esta tendencia. Los autores demuestran que los mecanismos de autoatención son inherentemente invariantes a la permutación, lo que provoca una pérdida de información sobre el orden temporal continuo de la señal. En consecuencia, estas redes altamente parametrizadas tienden a sobreajustarse y degradar el modelado de dependencias temporales estrictas frente a enfoques lineales simples que preservan intacta la cronología de los datos.

Por este motivo, se establece a DLinear no solo como un modelo predictivo objeto de estudio, sino conceptualmente como el *baseline* estructural frente al resto de arquitecturas avanzadas. Al reducir el proceso de inferencia a una mera extracción de tendencia por media móvil y un mapeo directo mediante una capa lineal, DLinear carece de la capacidad de extraer características complejas a múltiples escalas, al contrario que MICN, o de modelar interacciones cruzadas complejas, como sí hace la arquitectura Transformer.

## 3.6 Implementación del Flujo de Entrenamiento y Predicción

El presente capítulo detalla la arquitectura de software modular empleada para la ejecución automatizada de los modelos PyPOTS. En los apartados sucesivos se desgrena el tratamiento aplicado a las series temporales, el diseño del espacio de búsqueda, los mecanismos de entrenamiento y monitorización, y el marco de métricas establecido para evaluar el rendimiento predictivo final frente a los modelos estadísticos de referencia.

### 3.6.1 Preparación y Enmascaramiento de Datos

De forma previa a la instanciación de cualquier arquitectura neuronal, el sistema debe adecuar la información en bruto para hacerla compatible con ciertos requerimientos matemáticos. El proceso de ingesta inicia al cargar el conjunto de datos estructurado en formato *Parquet*. Con el objetivo de obtener unos tiempos de experimentación razonables con los recursos computacionales disponibles, se selecciona un subconjunto correspondiente a una fracción de los sensores IoT disponibles en la red urbana. Para asegurar la reproducibilidad estricta de todos los experimentos y aislar la varianza paramétrica, este muestreo se realiza tras fijar una semilla pseudoaleatoria constante.

A diferencia de otros problemas clásicos de aprendizaje automático, dividir series temporales exige respetar de forma estricta el orden cronológico. Una partición puramente aleatoria provocaría *data leakage* por lo que se segmenta el conjunto de datos en base a umbrales temporales exactos: el 60% inicial de los registros históricos conforma el conjunto de entrenamiento (*train*), el 20% intermedio se destina a la validación durante la optimización paramétrica (*validation*), y el 20% final se reserva de manera exclusiva para la evaluación definitiva del rendimiento predictivo (*test*).

Para transformar la serie temporal continua en una estructura espacial discreta que sea compatible con el entrenamiento por lotes de las redes neuronales, el componente de carga de datos implementa una técnica de ventanas deslizantes. Esta técnica se aplica diferentemente en función de la tarea objetivo:

- **Imputación.** El sistema extrae secuencias continuas avanzando un paso temporal por iteración. Para un tamaño de ventana  $W$ , el tensor resultante captura únicamente el contexto histórico local para ser reconstruido.



- **Forecasting.** La extracción se vuelve dual. Por cada iteración, el algoritmo recorta una matriz histórica  $X$  de tamaño  $W$  y, de forma contigua, extrae una matriz objetivo futura  $Y$  de longitud equivalente al horizonte predictivo predefinido. Esta separación estricta garantiza que el modelo proyecte el futuro sin contaminarse de los valores objetivo.

El resultado final de esta operación geométrica es la generación de tensores tsridimensionales de dimensiones  $(N, W, F)$ , donde  $N$  representa el número total de muestras extraídas de la ventana deslizante,  $W$  la profundidad temporal y  $F$  el número de características multivariantes seleccionadas (presencia de vehículos, coordenadas geoespaciales, componentes trigonométricas...).

Finalmente, la evaluación cuantitativa de los modelos de imputación exige disponer de una verdad fundamental (*ground truth*) para medir el margen de error con precisión matemática. Dado que el objetivo primordial consiste en predecir vacíos en la serie temporal, resulta indispensable ocultar de forma artificial una fracción de los datos reales durante la fase de entrenamiento.

Para los conjuntos de validación y prueba, el algoritmo identifica las coordenadas exactas de los valores correctamente observados y procede a enmascarar de forma estocástica un porcentaje del 15% de ellos con **NaNs**. Desde el punto de vista estadístico, este procedimiento genera un patrón de ausencia de tipo *Missing Completely At Random* (MCAR) [14], ya que la probabilidad de que una observación sea ocultada es totalmente independiente del estado del sistema. Esta estrategia matricial permite entregar una señal degradada a la red neuronal, mientras retiene la matriz original intacta. De este modo, el sistema analítico puede cuantificar el error absoluto y cuadrático de forma exclusiva sobre aquellas coordenadas espaciotemporales donde se introdujo el vacío artificial, lo cual garantiza un marco de medición objetivo e inalterable.

#### 3.6.2 Definición de Métricas de Evaluación

Para cuantificar el grado de precisión geométrica de las arquitecturas neuronales, resulta necesario establecer un marco matemático de evaluación robusto. Tanto en el contexto de la imputación de series temporales como en la predicción de las mismas, el sistema se enfrenta a un problema de minimización: cuanto menores resulten los valores de las métricas de error, mayor será el rendimiento del modelo. Este principio rige de igual manera en la fase de validación, para la selección de

hiperparámetros y parada temprana (*Early Stopping*), y en la fase de evaluación final en *test*.

La discrepancia entre los valores reales y las estimaciones generadas por la red neuronal se evalúa aislando la señal objetivo. En las tareas de imputación, el cálculo se restringe de forma exclusiva a las coordenadas donde se introdujo ruido sintético; mientras que, en la predicción, se evalúa sobre la totalidad de la ventana futura inexplorada. Si se define  $n$  como el número total de datos a evaluar,  $y_i$  como el valor real correspondiente a la verdad fundamental, y  $\hat{y}_i$  como la predicción generada por el modelo, el rendimiento del sistema se analiza a través de tres indicadores estandarizados.

La primera métrica implementada es el Error Absoluto Medio (MAE). Esta función calcula el promedio de las diferencias absolutas entre las predicciones y las observaciones empíricas. El MAE proporciona una medida lineal del error, lo que otorga el mismo peso a todas las desviaciones, independientemente de su magnitud, y facilita la interpretación directa en la misma escala que los datos originales. Matemáticamente, se expresa como:

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i| \quad (3.4)$$

Para penalizar de forma severa las discrepancias de gran magnitud, el sistema calcula el Error Cuadrático Medio (MSE). A diferencia de la métrica anterior, el MSE eleva al cuadrado la diferencia entre el valor estimado y el valor real antes de promediar los resultados. Esta propiedad matemática resulta fundamental para detectar y mitigar predicciones extremas u *outliers* generados por el modelo, por lo que se establece como la métrica principal para guiar el mecanismo de parada temprana durante el entrenamiento. Su formulación es la siguiente:

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 \quad (3.5)$$

Finalmente, para preservar la alta sensibilidad ante errores pronunciados del MSE, pero retornar el resultado a las unidades originales de la variable estudiada, se calcula la Raíz del Error Cuadrático Medio (RMSE). Esta métrica permite interpretar la desviación estándar de los residuos de las predicciones, lo que resulta de gran utilidad para comparar el rendimiento final de distintas familias algorítmicas en la fase de prueba. Su cálculo se obtiene al aplicar la raíz cuadrada a la función anterior:

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2} \quad (3.6)$$

### 3.6.3 Espacios de Búsqueda y Justificación Paramétrica

Dada la alta complejidad interna de los modelos empleados, se considera la realización de una optimización de hiperparámetros para refinar su comportamiento. En lugar de evaluar permutaciones infinitas, el sistema fusiona un conjunto de parámetros estructurales estáticos con un espacio de búsqueda dinámico acotado. Para evitar *overfitting*, el número de épocas no se somete a exploración; se establece un límite superior conservador (entre 20 y 30 iteraciones dependiendo de la tarea) acoplado a un mecanismo de parada temprana con una paciencia estricta de 2 épocas, configurado para almacenar de forma exclusiva los pesos obtenidos por los modelos con menor MSE en validación.

Para dotar a esta búsqueda de la mejor configuración de hiperparámetros de rigor estadístico, el número de iteraciones ejecutadas se fundamenta probabilísticamente. Matemáticamente, la probabilidad  $P$  de encontrar al menos una configuración paramétrica que se sitúe en el 10% superior del espacio de rendimiento tras  $n$  iteraciones independientes se define como:

$$P = 1 - (1 - 0.1)^n$$

Para garantizar un nivel de confianza empírica superior al 90% ( $P > 0.90$ ), se requiere resolver la inequación, lo que arroja un mínimo de  $n = 22$  iteraciones. Por consiguiente, el diseño experimental se establece en 22 iteraciones como estándar metodológico para todas las arquitecturas evaluadas.

### Configuraciones para tareas de Imputación

Para el modelo SAITS, el espacio de configuración final resultante de la fusión paramétrica se define como sigue:

```
^^I^^I{
^^I^^I^^I"epochs": 20,
^^I^^I^^I"patience": 2,
^^I^^I^^I"model_saving_strategy": "best",
^^I^^I^^I"window_size": [48, 96],
^^I^^I^^I"batch_size": [32, 64],
^^I^^I^^I"lr": {"distribution": "loguniform", "min": 0.0001,
    "max": 0.005},
^^I^^I^^I"n_layers": [2, 3],
^^I^^I^^I"d_model": [128, 256],
^^I^^I^^I"n_heads": [2, 4],
^^I^^I^^I"dropout": [0.1, 0.2]
^^I^^I}
```

---

Código 3.1: Espacio de búsqueda y configuración base para la arquitectura SAITS.

La selección teórica de estos umbrales responde a las particularidades topológicas de las series temporales. A diferencia de los modelos de procesamiento de lenguaje natural, las redes aplicadas a predicción secuencial tienden a memorizar el ruido local si se diseñan con excesiva profundidad [22]. Por este motivo, el número de capas se restringe deliberadamente a un rango superficial de entre 2 y 3 bloques. Asimismo, se opta por explorar tamaños de ventana temporal de 48 y 96 pasos, dimensiones diseñadas de forma específica para capturar ciclos locales (12 y 24 horas) en los sensores de tráfico urbano. En consonancia con la literatura original [19], la dimensionalidad latente se establece en potencias de 2 (128, 256) estrictamente divisibles por el número de cabezales de atención. La tasa de aprendizaje se explora bajo una distribución log-uniforme para permitir al optimizador muestrear magnitudes de ajuste fino con mayor frecuencia.

Por otro lado, la arquitectura probabilística de difusión (CSDI) presenta requerimientos computacionales notablemente distintos:

```
^^I^^I{
^^I^^I^^I"epochs": 20,
^^I^^I^^I"patience": 2,
^^I^^I^^I"model_saving_strategy": "best",
^^I^^I^^I"target_strategy": "random",
^^I^^I^^I"n_diffusion_steps": 30,
^^I^^I^^I"d_time_embedding": 128,
^^I^^I^^I"d_feature_embedding": 128,
^^I^^I^^I"d_diffusion_embedding": 128,
^^I^^I^^I"window_size": [48, 96],
^^I^^I^^I"batch_size": [64, 128],
^^I^^I^^I"lr": {"distribution": "loguniform", "min": 0.0001,
    "max": 0.005},
^^I^^I^^I"n_layers": [3, 4],
^^I^^I^^I"n_channels": [64, 128],
^^I^^I^^I"n_heads": [2, 4],
^^I^^I^^I"dropout": [0.1, 0.2]
^^I^^I}
```

---

Código 3.2: Espacio de búsqueda y configuración base para la arquitectura CSDI.

En este segundo escenario, las variables exclusivas del proceso estocástico no se someten a exploración aleatoria para evitar obtener configuraciones de parámetros entrenables enormes. Los *embeddings* se fijan en 128 dimensiones al seguir de forma rigurosa las especificaciones empíricas recomendadas por los autores originales [17]. Resulta crítico destacar el redimensionamiento de dos parámetros clave para viabilizar el entrenamiento: los pasos de difusión se acotan a 30 para obtener tiempos de inferencia razonables y el tamaño de lote se eleva a un rango de entre 64 y 128 muestras para maximizar la paralelización en la tarjeta gráfica.

### Configuraciones para Tareas de Predicción

A diferencia de la imputación local, el abordaje del problema predictivo exige a las redes neuronales asimilar un contexto histórico mucho más extenso. En problemas de predicción de series temporales multivariantes, resulta imperativo establecer una relación óptima entre la longitud de la ventana de observación histórica ( $L_{in}$ ) y el horizonte de predicción objetivo ( $L_{out}$ ).

En la presente investigación, para el escenario predictivo a corto plazo (24 horas), se ha adoptado la estrategia heurística estándar de la literatura en la que la ventana de entrada se establece como el doble del horizonte temporal. Dado que la red IoT registra eventos cada 15 minutos, el horizonte de 24 horas equivale a 96 pasos ( $L_{out}^{24h} = 96$ ) y consecuentemente requiere un contexto de entrada de 192 pasos ( $L_{in}^{24h} = 192$ ). Para el escenario a medio plazo (48 horas), preservar esta proporcionalidad estricta implica procesar un horizonte de 192 pasos a partir de un bloque histórico de 384 pasos ( $L_{in}^{48h} = 384$ ).

Para acometer esta tarea a gran escala, el modelo DLinear define su espacio de búsqueda de hiperparámetros como sigue:

---

```

^^I^^I{
^^I^^I^^I"epochs": 30,
^^I^^I^^I"patience": 2,
^^I^^I^^I"model_saving_strategy": "best",
^^I^^I^^I"window_size": [192, 384],
^^I^^I^^I"batch_size": [64, 128, 256],
^^I^^I^^I"lr": {"distribution": "loguniform", "min": 0.0005,
    ↵ "max": 0.01},
^^I^^I^^I"moving_avg_window_size": [25, 51, 99, 149]
^^I^^I}

```

---

Código 3.3: Espacio de búsqueda y configuración base para la arquitectura DLinear.

Dada la extrema ligereza matemática de sus capas completamente conectadas, el algoritmo DLinear admite tolerancias elevadas en la ingesta de datos. Esto permite que el tamaño de lote se escale hasta 256 muestras por iteración en ambos horizontes sin provocar desbordamientos de VRAM. El parámetro crítico de esta arquitectura radica en su núcleo de descomposición, una técnica de filtrado adoptada de arquitecturas predecesoras como Autoformer [21]: **la ventana de media móvil**. Se exploran dimensiones impares para garantizar un filtrado central simétrico, evitando desfases cronológicos. Es imperativo señalar la bifurcación estructural dependiente del horizonte: para el escenario de 24 horas (`window_size = 192`), el espacio exploró núcleos más cerrados (`[25, 51, 99]`), mientras que para el escenario de 48 horas (`window_size = 384`), el espacio debió ensancharse (`[51, 99, 149]`) para evitar que un filtro demasiado pequeño

resultara incapaz de capturar la macro tendencia en un vector histórico del doble de longitud.

En contraste, la parametrización de la arquitectura Transformer exige un diseño exploratorio mucho más conservador, debido a su complejidad algorítmica cuadrática:

---

```

^^I^^I{
^^I^^I^^I"epochs": 20,
^^I^^I^^I"patience": 2,
^^I^^I^^I"model_saving_strategy": "best",
^^I^^I^^I"window_size": [192, 384],
^^I^^I^^I"batch_size": [16, 32, 64, 128],
^^I^^I^^I"lr": {"distribution": "loguniform", "min": 0.0001,
    "max": 0.005},
^^I^^I^^I"n_layers": [2, 3],
^^I^^I^^I"d_model": [128, 256],
^^I^^I^^I"n_heads": [2, 4, 8],
^^I^^I^^I"dropout": [0.1, 0.2]
^^I^^I}

```

---

Código 3.4: Espacio de búsqueda y configuración base para la arquitectura Transformer.

Al someter el mecanismo de autoatención a secuencias continuas de hasta 384 pasos, el consumo de memoria escala exponencialmente debido a la complejidad espacial  $O(L^2)$  del *self-attention* estándar, una limitación crítica ampliamente documentada en el análisis de series temporales largas [23]. Esta limitación requiere una adaptación asimétrica del espacio de búsqueda en función del tamaño de ventana. Mientras que para la ventana de 192 pasos la tarjeta gráfica admite tamaños de lote convencionales (**batch\_size** entre 32 y 128 secuencias), al procesar la ventana de 384 pasos el lote máximo admitido antes del desbordamiento de memoria se desploma a 64, por lo que se incluye exploración en rangos mínimos operativos de hasta 16 secuencias. En ambas configuraciones, el espacio prioriza la diversificación de la dimensionalidad latente y la multiplicidad de cabezales, con el objetivo de dotar al modelo de la máxima capacidad de abstracción bajo estas restricciones de hardware.

Finalmente, fiel a los preceptos teóricos que la postulan como una alternativa espacialmente más ligera que los Transformers [20], se establece la configuración de la arquitectura convolucional isométrica (MICN):

```

^^I^^I{
^^I^^I^^I"epochs": 20,
^^I^^I^^I"patience": 2,
^^I^^I^^I"model_saving_strategy": "best",
^^I^^I^^I"conv_kernel": [7, 9],
^^I^^I^^I"window_size": [192],
^^I^^I^^I"batch_size": [32, 64, 128],
^^I^^I^^I"lr": {"distribution": "loguniform", "min": 0.0005,
    "max": 0.005},
^^I^^I^^I"n_layers": [1, 2],
^^I^^I^^I"d_model": [64, 128],
^^I^^I^^I"dropout": [0.1, 0.2]
^^I^^I}

```

Código 3.5: Espacio de búsqueda y configuración base para la arquitectura MICN.

Como se documenta en el espacio paramétrico (`window_size = 192`), el diseño experimental de MICN rompe con la proporcionalidad estricta descrita al inicio de la sección, manteniendo un espacio de búsqueda unificado y estático para ambos horizontes predictivos (24 y 48 horas). La literatura empírica demuestra que el escalado lineal no siempre es computacionalmente viable. En arquitecturas basadas en convoluciones multiescala y submuestreo jerárquico como MICN, inyectar ventanas de 384 pasos provoca el colapso del campo receptivo, derivando en inestabilidades matriciales donde el tamaño del filtro supera la dimensión de la señal comprimida en las capas profundas.

Por este motivo estructural, para el escenario de 48 horas se ha optado metodológicamente por igualar la ventana de entrada al horizonte de predicción ( $L_{in}^{48h} = L_{out}^{48h} = 192$ ). Esta decisión garantiza la estabilidad numérica necesitada para las convoluciones de la red.

Esta homogeneidad espacial de entrada, combinada con la complejidad computacional lineal  $O(L)$  de la arquitectura, permite explorar tamaños de lote consistentes y amplios (`batch_size` entre 32 y 128 secuencias) en ambas pruebas sin



riesgo de desbordamiento. Para preservar el propósito fundacional de sus creadores, el espacio acota deliberadamente la dimensionalidad latente (`d_model` en 64 y 128) y restringe la profundidad a redes superficiales (`n_layers` a 1 o 2 bloques). Finalmente, su núcleo de extracción se apoya en una parametrización convolucional estática (`conv_kernel: [7, 9]`), donde el filtro menor captura las dependencias locales de alta frecuencia y el superior modela las macro-tendencias globales de la señal.

### 3.6.4 Orquestación y Automatización del Flujo

El proceso de entrenamiento se automatiza completamente mediante una arquitectura de software diseñada para aislar la complejidad matemática y maximizar el uso de los recursos computacionales de forma desatendida. Para ello se desarrollan *scripts* de *Bash* para ejecutar secuencialmente las diferentes iteraciones de los modelos de imputación y predicción. Estos archivos actúan como una capa de ejecución superior del entorno: se inicializa el entorno virtual, se inyecta las variables necesarias para el enlace correcto de los módulos y se lanzan los comandos de ejecución del pipeline de optimización. Esta aproximación de cola secuencial garantiza la estabilidad física de los recursos y permite ejecutar ciclos completos de optimización de manera ininterrumpida.

Una vez el *script* inicia la ejecución de un modelo, el ciclo de vida algorítmico sigue un esquema lógico iterativo, el cual se ilustra detalladamente en la Figura 3.5.

El sistema orbita en torno a un orquestador central que gestiona este flujo. El proceso se divide en las siguientes etapas fundamentales:

1. **Carga de entorno e inicialización:** El sistema procesa los diccionarios de hiperparámetros y prepara los espacios de búsqueda dinámica definidos en la Sección 3.6.3.
2. **Generación de configuración.** En cada iteración, el algoritmo genera una configuración paramétrica única mediante un muestreo probabilístico (*Random Search*). Si el espacio requiere una distribución log-uniforme (como la tasa de aprendizaje), se transforma el intervalo matemáticamente para extraer un valor continuo aleatorio.
3. **Entrenamiento y Evaluación.** El orquestador delega el flujo en el módulo de entrenamiento, el cual inicializa la red neuronal. Durante esta fase, se evalúa la capacidad de generalización del modelo utilizando los criterios de

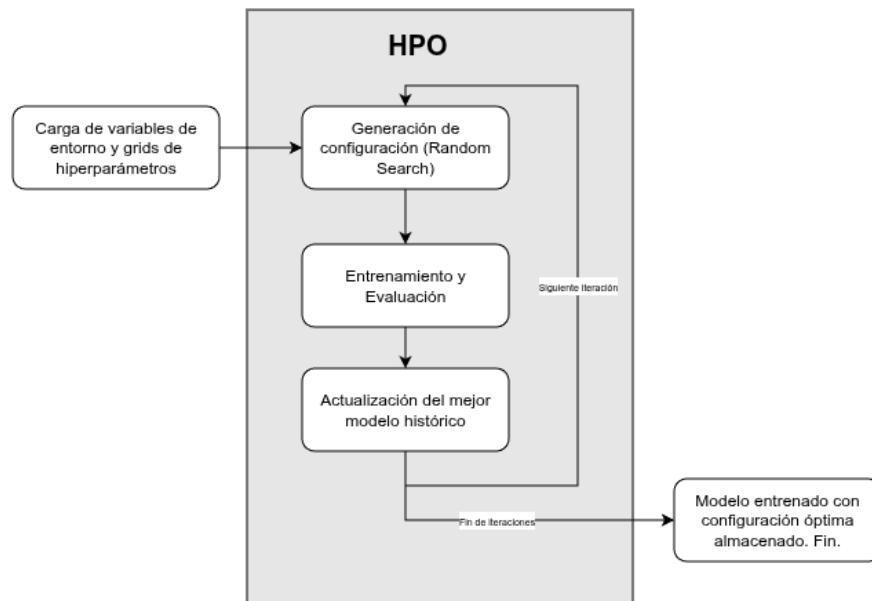


Figura 3.5: *High Performance Optimization*: flujo lógico de optimización de hiperparámetros.

validación y las métricas de error previamente definidas (ver Sección 3.6.2). Además, para dotar al proceso de robustez, toda invocación se encapsula en un bloque lógico de control de excepciones, de forma que se contemplen posibles errores de CUDA y/o GPU. Si una iteración falla, se libera la caché de la GPU y se avanza a la siguiente iteración sin interrumpir el flujo global.

4. **Actualización del mejor modelo histórico.** Tras evaluar la iteración, el orquestador compara la métrica de error resultante con el mejor registro histórico. Si la configuración actual supera a la anterior, se retienen en memoria y en disco los nuevos pesos óptimos.

El ciclo se repite hasta alcanzar el límite de pruebas (*trials*) estipulado en el archivo *Bash*. Al finalizar, el algoritmo consolida un registro estructurado resumiendo la configuración ganadora, dejando el modelo óptimo preparado para la fase final de evaluación predictiva.

Por último, cabe comentar que el sistema integra capacidades de telemetría directa hacia TensorBoard para dotar a la fase de entrenamiento de observabilidad empírica. Durante cada época de ajuste, el algoritmo registra la evolución de las funciones de pérdida y las métricas de validación. Esto resulta especialmente útil para el posterior análisis detallado de la curva de aprendizaje y la evolución del *score*.

### 3.6.5 Evaluación Final en Test

Una vez concluida la fase de optimización paramétrica, se evalúan las arquitecturas entrenadas con la mejor configuración de hiperparámetros sobre el conjunto de *test*. Para ello, el componente de carga de datos instancia la partición de prueba y los pesos almacenados de los modelos que mejor métricas han obtenido. Posteriormente se inicia la evaluación, que se implementa diferentemente para los modelos de imputación y predicción:

- **Imputación.** El margen de error se calcula aislando de forma exclusiva las coordenadas espaciotemporales con **NaNs** sintéticos inyectados. De forma contigua, se invocan de manera determinista los estimadores estadísticos de referencia (Media Global y LOCF) bajo las mismas condiciones para establecer la cota inferior de rendimiento aceptable.
- **Predicción.** El rendimiento se evalúa frente a la totalidad de la ventana futura predefinida, tras descartar los valores nulos propios de la serie temporal original, puesto que no se pueden evaluar predicciones sobre valores desconocidos.

### Evaluación Integrada: Imputación + Predicción

Como culminación de la arquitectura de software, el sistema implementa un flujo algorítmico adicional de evaluación integrada (*pipeline*) que fusiona secuencialmente los dos subdominios estudiados. El objetivo empírico de este conducto es realizar un ensayo comparativo para cuantificar si las arquitecturas de *forecasting* logran incrementar su rendimiento predictivo al recibir un contexto histórico reconstruido matemáticamente, en lugar de procesar los tensores crudos con las anomalías naturales de la red sensorica.

Para asegurar una metodología comparativa rigurosa, el módulo evaluador despliega el experimento en dos iteraciones de inferencia sobre el mismo conjunto de prueba (*test set*). En la primera iteración, la red neuronal de *forecasting* ingiere la serie temporal inalterada (evaluación estándar en test) y emite sus predicciones. Acto seguido, se activa la segunda iteración, que requiere la instanciación simultánea de un modelo de imputación y de la red de predicción.

Dado que los modelos de predicción pueden operar sobre ventanas históricas diferentes de las empleadas por el imputador con el que se combinan, se detecta el factor de divisibilidad entre ambas arquitecturas y se secciona la ventana más

amplia en sub-bloques continuos. Tras realizar el proceso de imputación sobre estos subtensores, se ensamblan matricialmente de nuevo a su formato original. El resultado es una matriz histórica continua que sirve como entrada para realizar el *forecasting*.

Al igual que se ha realizado en la etapa de entrenamiento previa, este pipeline también se ejecuta desde un *script* de *bash*, de forma que se realice una evaluación ininterrumpida de múltiples combinaciones interesantes a analizar.

## 3.7 Diseño del Pipeline ETL en tiempo real

En esta sección se detalla la arquitectura desarrollada para el consumo y almacenamiento automatizado de información en tiempo real. Lejos de constituir un mero ejercicio de recolección de datos, el despliegue ininterrumpido de este flujo durante un periodo de un mes responde a una necesidad metodológica fundamental en el paradigma de operaciones de MLOps: la validación *Out-of-Time* (OOT). La ingesta masiva de información permite construir un conjunto de datos actualizado y desvinculado temporalmente del *dataset* histórico con el que se entrenaron los modelos. De este modo, se puede auditar la robustez de los algoritmos ante nuevos datos y medir la degradación predictiva frente a la deriva conceptual inherente a la evolución del tráfico urbano.

El código se estructura de forma modular para separar la lógica de conexión a la API, la manipulación de los datos y la persistencia en la base de datos.

### 3.7.1 Lógica de consumo y esquema de almacenamiento de datos

El primer eslabón del *pipeline* consiste en la ingesta de información desde el *endpoint* público proporcionado por la ciudad de Melbourne. Para ello, se desarrolla un módulo de conexión que encapsula las peticiones HTTP.

La extracción se realiza mediante un sistema de paginación controlada, de forma que se respeten estrictamente las restricciones de consumo impuestas por el proveedor de la API. Se definen filtros específicos que solicitan bloques de 100 registros ordenados desde la última actualización.

La fase de carga consolida la información consumida en un entorno analítico (OLAP) basado en DuckDB. A diferencia de los sistemas transaccionales tradicionales, este motor de base de datos está diseñado para maximizar el rendimiento en consultas de lectura masiva y agregación de métricas. Por ello, para optimizar la ingesta desde Apache Airflow y el consumo posterior por parte de los modelos, se opta por un diseño de tabla desnormalizada.

Al centralizar la información en una única entidad, se eliminan las costosas operaciones de cruce (*JOIN*) durante las fases de procesamiento y evaluación predictiva. La estructura de esta entidad principal, generada a partir del volcado directo de la API en tiempo real, se detalla a continuación mediante su diccionario de datos (ver Tabla 3.1).

Tabla 3.1: Diccionario de datos en DuckDB.

| Nombre de Columna               | Tipo de Dato         | Descripción                                                                                                                           |
|---------------------------------|----------------------|---------------------------------------------------------------------------------------------------------------------------------------|
| <code>kerbsideid</code>         | <code>BIGINT</code>  | Identificador físico único de la plaza de aparcamiento o sensor.                                                                      |
| <code>status_description</code> | <code>VARCHAR</code> | Estado actual de la plaza de aparcamiento (e.g., <i>Present</i> , <i>Unoccupied</i> ). Constituye la base para la variable objetivo.  |
| <code>status_timestamp</code>   | <code>VARCHAR</code> | Marca temporal exacta en la que el sensor detectó el cambio de estado.                                                                |
| <code>lastupdated</code>        | <code>VARCHAR</code> | Marca temporal de la última sincronización del registro con el sistema central.                                                       |
| <code>zone_number</code>        | <code>DOUBLE</code>  | Identificador numérico de la zona tarifaria o sector de estacionamiento.                                                              |
| <code>location</code>           | <code>STRUCT</code>  | Estructura de datos anidada que contiene las coordenadas geográficas exactas ( <code>lat</code> y <code>lon</code> ) del dispositivo. |

Para automatizar la creación de tablas sin necesidad de definir sentencias DDL (*Data Definition Language*) estáticas, se aprovecha una característica de interoperabilidad nativa entre DuckDB y Pandas. El sistema ejecuta una consulta inicial con la condición `WHERE 1=0`. Al evaluarse como falsa, DuckDB no inserta filas, pero copia y adapta instantáneamente la estructura de columnas y tipos de datos del *DataFrame* de Pandas para inicializar la tabla si esta no existe.

Resulta de vital importancia considerar que el acceso a la base de datos se encuentra encapsulado en un bloque `try...finally`, garantizando que la desconexión se ejecute de forma determinista y liberando el archivo físico de la escritura incluso ante la ocurrencia de una excepción inesperada.

## 3.7.2 Orquestación del flujo

Para cohesionar las fases descritas, se implementa un grafo acíclico dirigido (DAG) en Apache Airflow, el cual permite definir las dependencias del *pipeline* como funciones nativas de Python. De forma secuencial, se realiza primero el consumo de datos y, a continuación, el almacenamiento persistente. Airflow gestiona implícitamente la transferencia de metadatos y del *payload* JSON entre las tareas empleando su sistema de mensajería interna (XCom). La topología resultante de este flujo se ilustra en la Figura 3.6.

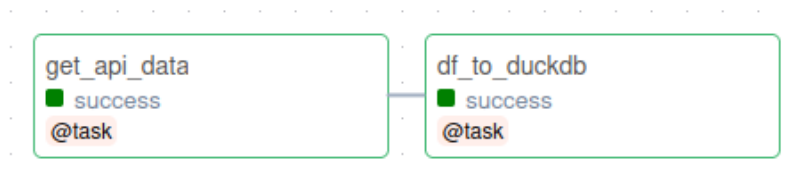


Figura 3.6: Representación visual del flujo de tareas (DAG) en la interfaz de Apache Airflow.

El flujo se configura para ejecutarse de manera autónoma con una periodicidad cronológica mediante el parámetro `schedule='*/15 * * * *`', disparando una nueva ejecución de extracción, transformación y carga cada 15 minutos. El parámetro `catchup=False` asegura que, en caso de una caída del servidor, Airflow no intente ejecutar retrospectivamente todos los intervalos perdidos de forma simultánea, evitando así una violación de las políticas de uso de la API por exceso de peticiones.

## 3.8 Despliegue en Producción

En esta sección se detalla el proceso de empaquetado, configuración y despliegue del *pipeline* ETL en un entorno de servidor remoto. La transición desde un entorno de desarrollo local hacia una infraestructura de producción resulta indispensable

para garantizar la alta disponibilidad del sistema y la ingesta ininterrumpida de datos.

#### 3.8.1 Arquitectura del sistema en Docker

Como se comenta anteriormente, para asegurar que los procesos de recolección de datos asíncronos operen sin interrupciones, el sistema se despliega en un servidor remoto. El acceso y desarrollo sobre este servidor se realiza mediante el protocolo *Secure Shell* (SSH) a través del entorno de Visual Studio Code.

Con el fin de evitar problemas de incompatibilidad de entornos, se opta por una arquitectura basada en contenedores utilizando *Docker* y *Docker Compose*. Se empaqueta el código junto con sus dependencias para garantizar que el sistema sea reproducible en cualquier servidor operativo, independientemente de su configuración base.

En relación con la configuración de Apache Airflow, se realiza una adaptación de la infraestructura oficial a las necesidades y recursos específicos del proyecto:

- **Construcción de imagen personalizada.** Durante las pruebas de despliegue, la instalación dinámica de dependencias ha presentado conflictos de permisos al requerir compiladores del sistema (como `g++` para la compilación nativa de DuckDB). Para solventarlo, se diseña un **Dockerfile** propio que parte de la imagen base de Airflow: se instalan las herramientas del sistema operativo como superusuario (`root`) y, posteriormente, las librerías de Python de forma segura.
- **Mapeo de Volúmenes.** Se mapean los directorios locales del repositorio hacia el interior del contenedor. El directorio de código fuente se enruta a `/opt/airflow/plugins/src`, la base de datos DuckDB a `/opt/airflow/duck_db`, y el archivo orquestador (`dag.py`) a la carpeta oficial `/opt/airflow/dags/` para su detección automática por el planificador.
- **Control de ejecución (`.airflowignore`).** Para evitar que el servidor web de Airflow procese código ajeno a la orquestación, se implementa un archivo `.airflowignore`. Esto restringe el escaneo a los módulos pertinentes para garantizar la estabilidad del contenedor.

Por otro lado, la monitorización del *pipeline* requiere acceso a la interfaz gráfica (UI) de Airflow, la cual opera por defecto en el puerto 8080. Para maximizar la seguridad de la infraestructura y mitigar posibles vulnerabilidades, se decide no exponer dicho puerto en el *firewall* público del servidor remoto. En su lugar, se implementa una estrategia de acceso mediante *Port Forwarding* a través del túnel SSH. De este modo, se enruta el tráfico de red de forma cifrada desde el servidor hasta la máquina local.

### 3.8.2 Lanzamiento y Monitorización

Una vez configurados los parámetros del contenedor, el despliegue se inicializa mediante el comando `docker compose up -d`. La utilización del modo *detached* permite que los servicios subyacentes de Airflow (planificador, servidor web, *workers* y base de datos interna) se ejecuten en segundo plano.

Como resultado de este despliegue, el planificador de Airflow asume el control del ciclo de vida del *pipeline*. En la Figura 3.7 se presenta el historial de monitorización del sistema en producción, de forma que se evidencia la estabilidad de la ingesta recurrente cada 15 minutos sin intervención manual.

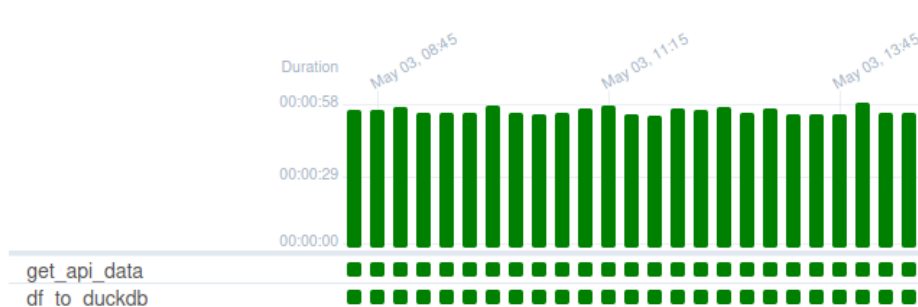


Figura 3.7: Monitorización de las ejecuciones recurrentes del *pipeline* ETL en el entorno de producción.

### 3.8.3 Finalización de la ingesta y consolidación de datos

Tras mantener el consumo de datos de forma ininterrumpida durante un mes, rango temporal suficiente para poder probar los modelos ya entrenados, se procede a



la interrupción deliberada del sistema. Para garantizar la integridad de la base de datos y evitar problemas en el archivo físico, el proceso de desconexión se ejecuta en tres fases secuenciales:

1. Se pausa la ejecución del grafo (DAG) directamente desde la interfaz gráfica de Apache Airflow, para evitar que el planificador dispare nuevas peticiones HTTP asíncronas a la API de Melbourne.
2. Se detienen y destruyen los contenedores del servidor para asegurar un cierre limpio del motor de base de datos interno.
3. Se realiza una copia de seguridad del archivo físico desde el servidor remoto hacia la máquina local, mediante protocolos de transferencia segura sobre el túnel SSH establecido.

## 3.9 Transformación del conjunto de datos en tiempo real

Una vez consolidado el archivo físico de la base de datos en el entorno local, resulta imperativo transformar los registros crudos obtenidos de la API para que coincidan de forma exacta con la topología tensorial exigida por los modelos neuronales entrenados. El desafío técnico principal de esta fase radica en la naturaleza operativa de las APIs de los dispositivos emisores IoT, debido a que estos sistemas no emiten datos en un flujo constante, sino que se rigen por una arquitectura *event-driven*: si el estado físico de una plaza no varía durante un periodo prolongado, el sistema no genera nuevas observaciones.

Este comportamiento es característico de las series temporales basadas en eventos, en las que la dinámica subyacente del sistema es continua, pero la observación es esparsa y dependiente de los cambios de estado [11]. En consecuencia, la aplicación directa de modelos de aprendizaje profundo exige la construcción de una representación temporal regular mediante técnicas de remuestreo.

### 3.9.1 Análisis estadístico de la dinámica temporal del sistema

Con el objetivo de caracterizar esta naturaleza esparsa, se realiza un análisis exploratorio de la distribución de los intervalos entre eventos consecutivos por sensor. Este análisis permite evaluar la regularidad del flujo de datos y cuantificar la presencia de interrupciones en la transmisión de la información.

Los resultados demuestran que la mediana de los intervalos entre eventos se sitúa en el orden de minutos, mientras que el percentil 95 se encuentra en torno a varias horas, lo cual se alinea con la rotación habitual de vehículos en un entorno urbano. No obstante, se observan casos en la cola de la distribución temporal que presentan intervalos de varios días de duración e incluso semanas. Al no disponer de etiquetas explícitas de fallo emitidas por el sistema central, resulta matemáticamente imposible distinguir de forma determinista entre la inmovilidad prolongada de un vehículo y un fallo del dispositivo. Consecuentemente, la aplicación de cualquier umbral estricto basado en la duración del intervalo para la detección de sensores defectuosos introduciría supuestos empíricos no verificables y potencialmente sesgados. Por este motivo, el presente trabajo descarta la inferencia explícita de fallos a partir del truncamiento de la distribución de intervalos temporales.

### 3.9.2 Procesamiento y reconstrucción de las señales

Para resolver la disparidad estructural y generar la matriz densa requerida por las redes neuronales, en primer lugar se eliminan los eventos duplicados y se genera un vector de tiempo continuo con una frecuencia estricta de 15 minutos, de forma análoga al proceso detallado en la sección 3.4. Al reindexar el conjunto de datos original sobre esta nueva cuadrícula homogénea, se generan vacíos explícitos en los intervalos sin eventos.

Dado que el estado de ocupación de una plaza de aparcamiento presenta una alta propiedad de persistencia temporal, se adopta la hipótesis de *Zero-Order Hold* (ZOH), ampliamente utilizada en sistemas dinámicos y procesamiento de señales telemáticas [3]. Según este principio, el último estado observado se mantiene constante y válido hasta la llegada de una nueva observación que certifique lo contrario. Bajo este supuesto, los huecos generados tras el re-muestreo son reconstruidos íntegramente mediante un proceso de *forward fill*.

Una vez completada la matriz continua, el proceso de estructuración tensorial y enriquecimiento trigonométrico es matemáticamente idéntico al desarrollado para el conjunto de datos histórico (véase la sección 3.6.1), así como la técnica de ventanas deslizantes aplicada. No obstante, se deshabilita la generación de diferentes particiones, pues todos los datos se emplean en el mismo conjunto de *test*.

Se adoptan las dos aproximaciones homólogas diseñadas en el marco experimental previo:

1. **Evaluación de imputación:** se evalúa mediante el mismo esquema de ausencia artificial (MCAR) descrito en la Sección 3.6.1. Sobre la serie temporal previamente completada mediante ZOH, se enmascara de forma estocástica una proporción idéntica de las observaciones (e.g., 15%). Esta inyección de ruido permite evaluar la capacidad de reconstrucción del modelo frente a caídas de red bajo escenarios donde existe un *ground truth* absoluto [5].
2. **Evaluación de predicción futura:** se ejecuta directamente sobre la serie temporal reconstruida mediante *forward filling*, sin enmascaramiento.

## 3.10 Clasificación espacial de zonas de estacionamiento

La dinámica de ocupación y rotación de los estacionamientos urbanos está intrínsecamente ligada al entorno geográfico que los rodea. Dotar a cada sensor de un contexto espacial adicional, de forma complementaria a las coordenadas geográficas, simplifica la comprensión sobre cómo fluctúa la demanda de los mismos. En consecuencia, se ha desarrollado un módulo específico encargado de clasificar el entorno urbano de cada plaza de aparcamiento y de generar una representación visual interactiva.

### 3.10.1 Extracción de puntos de interés (POIs)

Para determinar la naturaleza urbana del entorno de cada sensor, se utiliza la librería **OSMnx**, la cual permite interactuar directamente con la API pública de *OpenStreetMap* (Overpass API). El flujo de ejecución parte de la lectura del conjunto de datos geolocalizados, generado previamente durante la fase del EDA tras el cruce del histórico con el inventario geoespacial de la ciudad.

Para cada sensor, el algoritmo delimita un radio de búsqueda estricto de 100 metros. Esta distancia se ha calibrado para capturar únicamente la calle e inmediaciones directas del aparcamiento, evitando sesgos por calles colindantes de diferente naturaleza. Dentro de este radio, se extraen y contabilizan elementos clave de la infraestructura urbana filtrando por etiquetas específicas (*tags*): instalaciones públicas (*amenities*), tiendas (*shops*), oficinas (*offices*) y tipología de edificios (*building: residential, apartments*).

Para garantizar la estabilidad del proceso y respetar los límites de la API pública, se implementa una estrategia de uso de memoria caché local y pausas controladas (*rate limiting*) en las peticiones.

### 3.10.2 Algoritmo de clasificación ponderada

El principal reto técnico en la clasificación espacial es la existencia de zonas de uso mixto, especialmente en el centro de la ciudad (*Hoddle Grid* de Melbourne). En esta zona conviven rascacielos de oficinas, bajos comerciales y bloques de apartamentos residenciales.

Para solucionarlo, se implementa un sistema de puntuación ponderada. El algoritmo asigna un peso matemático a cada elemento geográfico en función de su impacto en el tráfico urbano. Definimos las variables correspondientes al recuento de puntos de interés en el radio de búsqueda como: *O* (oficinas), *S* (comercios minoristas o *shops*), *A* (instalaciones genéricas o *amenities*) y *R* (edificios residenciales).

Con base en estas variables, se computan dos métricas o cargas principales:

- **Carga Comercial ( $C_c$ ):** otorga un peso alto a las oficinas debido a la gran afluencia diaria de trabajadores, un peso medio a los comercios minoristas

y un peso unitario a las instalaciones genéricas.

$$C_c = 5O + 3S + A \quad (3.7)$$

- **Carga Residencial ( $C_r$ ):** asume que un solo polígono clasificado como edificio residencial alberga múltiples viviendas individuales, por lo que se le aplica un factor multiplicador elevado.

$$C_r = 10R \quad (3.8)$$

Posteriormente, el algoritmo evalúa y compara ambas cargas mediante una función definida a tramos para establecer la categoría espacial dominante ( $Z$ ) de la vía:

$$Z = \begin{cases} \text{Center (CBD)}, & \text{si } C_c > 120 \vee O \geq 10 \\ \text{Commercial}, & \text{si } (C_c > C_r \wedge C_c > 20) \vee C_c > 10 \\ \text{Residential}, & \text{si } C_r > C_c \wedge R > 0 \\ \text{General}, & \text{en cualquier otro caso} \end{cases} \quad (3.9)$$

Esta formulación permite que si la carga comercial es extrema o supera un umbral crítico de densidad de oficinas, la zona se clasifique inequívocamente como centro financiero. Si existe competencia entre ambos usos, se impone la carga dominante, dejando la etiqueta *General* como *fallback* para zonas de baja densidad. Finalmente, todos los sensores clasificados se registran en un log de control (`pois_sensors.log`) para facilitar su posterior auditoría y seguimiento.

### 3.10.3 Renderizado cartográfico e integración Web

Una vez enriquecido el conjunto de datos con la nueva categorización espacial, la última etapa del flujo consiste en su representación gráfica mediante la librería **Folium**.

Para maximizar el contraste visual de las distintas zonas, el mapa se inicializa centrado en las coordenadas del CBD de Melbourne empleando el mosaico oscuro *CartoDB dark\_matter*. El algoritmo itera sobre el conjunto de sensores clasificados, proyectando un marcador circular sobre el plano para cada uno de ellos. A cada categoría urbana se le ha asignado un código de color específico, tal y como

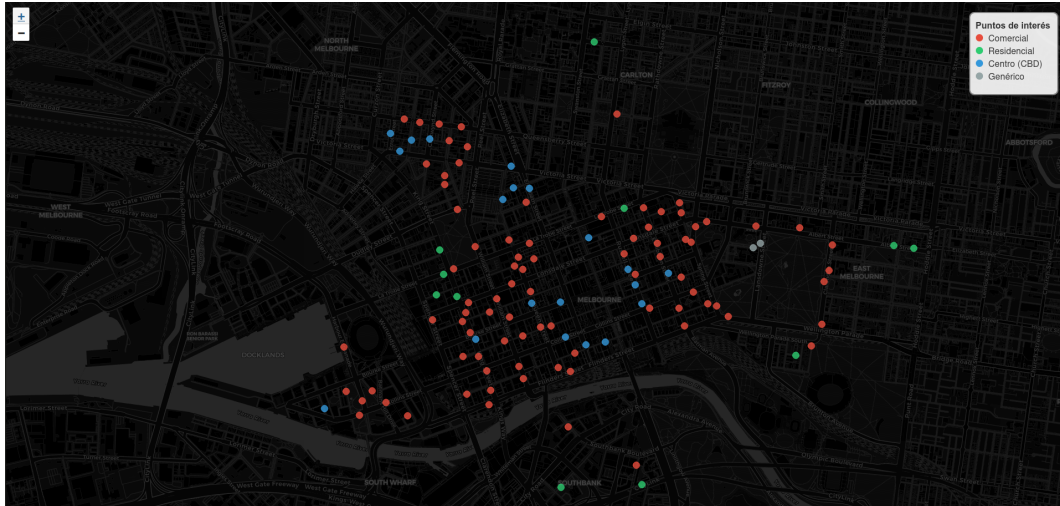


Figura 3.8: Distribución geográfica de las plazas de aparcamiento en Melbourne según su tipología urbana.

se ilustra en la Figura 3.8. Además, la visualización incorpora una leyenda descriptiva fijada mediante elementos HTML personalizados y un sistema de *tooltips* interactivos.

Por otro lado, al posicionar el cursor sobre un sensor específico, la interfaz revela en tiempo real su identificador único, el nombre de la vía a la que pertenece y la clasificación espacial que el algoritmo le ha asignado. Se puede apreciar esta funcionalidad en la Figura 3.9,

Finalmente, este mapa interactivo se exporta en formato HTML puro. Esta arquitectura de exportación permite que el mapa sea embebido de forma directa y fluida dentro de la página web documental del proyecto generada mediante **Quarto** sin requerir la ejecución local del código.

## 3.11 Desarrollo del portal de resultados con Quarto

Como culminación del flujo de procesamiento y modelado, se ha desarrollado un portal web interactivo destinado a la divulgación de los resultados del proyecto. Este entorno, construido íntegramente mediante el marco de publicación científica Quarto, permite trasladar la complejidad técnica del *pipeline* de datos y de las redes neuronales a un formato accesible, navegable y visualmente atractivo.

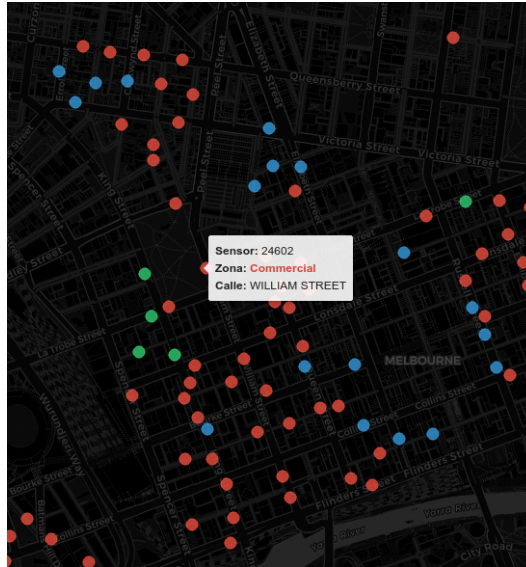


Figura 3.9: Detalle de la interfaz interactiva (*tooltip*) al inspeccionar un sensor individual.

#### 3.11.1 Configuración

La orquestación del portal se gestiona a través de un archivo de configuración centralizado (`_quarto.yml`). Se ha seleccionado una arquitectura de sitio web (`type: website`) renderizada en formato HTML5. Para el diseño de la interfaz de usuario (UI), se ha implementado el tema *Cosmo*, el cual proporciona una estética limpia y minimalista, complementada con hojas de estilo personalizadas (`styles.css`).

Para optimizar la experiencia de usuario (UX) y la legibilidad de un proyecto con alta carga de programación, se han activado dos características fundamentales a nivel global:

- **Índice de navegación dinámico (`toc: true`).** Genera automáticamente un menú lateral basado en la jerarquía de encabezados, facilitando el salto rápido entre secciones analíticas.
- **Plegado de código (`code-fold: true`).** Permite mantener una interfaz limpia al ocultar por defecto los bloques de código fuente (como los *scripts* de Python) a la vez que ofrece al usuario la opción de desplegarlos interactuando con la interfaz si desea auditar la lógica subyacente.

### 3.11.2 Navegación

El contenido del portal se ha estructurado jerárquicamente a través de una barra de navegación principal (*Navbar*), segmentando el proyecto en tres áreas clave de conocimiento:

1. **Inicio (`index.qmd`).** Actúa como página de aterrizaje (*landing page*) del proyecto. Presenta el contexto institucional mediante la inclusión gráfica del logotipo de la universidad, define el problema de la congestión del tráfico de agitación en Melbourne y resume la arquitectura metodológica en cuatro fases clave: ingesta analítica en DuckDB, procesamiento de series temporales, modelado mediante *Deep Learning* (PyPOTS) y divulgación.
2. **Visualizaciones (`visualizations.qmd`).** Espacio destinado a los resultados del Análisis Exploratorio de Datos (EDA) espacial y temporal. En esta sección se exponen los patrones descubiertos, integrando representaciones estáticas (como la detección de áreas de alta demanda o *hotspots*) y mapas interactivos generados con Folium.
3. **Resultados y Predicciones (`predictions.qmd`).** El núcleo evaluativo del modelo, donde se contrasta empíricamente el rendimiento de la red neuronal frente al *ground truth* de los sensores.

### 3.11.3 Ejecución Dinámica y Representación Gráfica

El principal valor añadido de utilizar Quarto frente a un generador de sitios estáticos tradicional es su integración nativa con el motor de ejecución de *Jupyter*. En lugar de importar imágenes estáticas pre-generadas, la página de resultados incluye bloques de código ejecutable que renderizan la información en el momento de la compilación del sitio.

A modo de ejemplo, para la evaluación visual del modelo PyPOTS, se embeben *scripts* de Python utilizando la librería `Matplotlib`. Mediante directivas internas de Quarto (como `#| label` y `#| fig-cap`), el código procesa los tensores con la evolución horaria de la ocupación real medida por los sensores frente a los volúmenes predichos por la red neuronal, trazando comparativas dinámicas.

Esta arquitectura asegura el paradigma de **investigación reproducible**: el documento final y el código que genera sus gráficos conviven en el mismo entorno fuente, garantizando que cualquier actualización en los pesos del modelo o en



la base de datos DuckDB se refleje automáticamente en la web con una simple recompilación del proyecto.

## 4 Experimentación y resultados

El presente capítulo expone de forma pormenorizada la fase de validación empírica y el análisis de resultados de las arquitecturas neuronales implementadas. Atendiendo a los objetivos de la investigación, la experimentación se diseña para validar la eficacia matemática de los modelos frente a dinámicas complejas del tráfico urbano. A lo largo de las siguientes secciones, se detallan las restricciones operativas del entorno, se analiza la convergencia de los modelos durante la fase de entrenamiento y se exponen las métricas obtenidas sobre el conjunto de datos inexplorado (*test*).

### 4.1 Restricciones experimentales

Previamente al análisis cuantitativo de los resultados, se definen ciertas consideraciones y restricciones de *hardware* que han moldeado la ejecución de los experimentos. La optimización de arquitecturas de *Deep Learning* sobre series temporales multivariantes exige establecer un balance realista entre el rigor metodológico teórico y la viabilidad computacional.

#### 4.1.1 Restricciones en la exploración paramétrica para CSDI

Como se ha definido anteriormente en el diseño arquitectónico, el sistema implementa una búsqueda aleatoria probabilística configurada a 22 iteraciones para garantizar un intervalo de confianza estadístico superior al 90% en el hallazgo de hiperparámetros óptimos. Sin embargo, cabe destacar una excepción a esta premisa teórica debido al extremo coste temporal observado durante el entrenamiento del modelo de imputación CSDI. Al tener que realizarse una inversión del ruido estocástico a través de cadenas de Márkov, la ejecución de 22 iteraciones

no es asumible con la infraestructura computacional disponible. En consecuencia, la exploración para CSDI se limita a 8 pruebas, lo cual supone un *trade-off* importante: se reduce significativamente la probabilidad de obtener la mejor configuración en favor de viabilizar la ejecución del experimento en tiempos razonables.

### 4.1.2 Ajuste de tamaño de ventana para MICN

En los experimentos de predicción es necesario establecer una relación coherente entre el tamaño de ventana de observación histórica ( $L_{in}$ ) y el horizonte de predicción objetivo ( $L_{out}$ ). Para los escenarios a corto plazo (predicción a 24 horas), se establece que la ventana de entrada es del doble de tamaño que el horizonte temporal. Dado un muestreo de 15 minutos ( $L_{out}^{24h} = 96$  pasos), la entrada se configura con 192 pasos ( $L_{in}^{24h} = 192$ ). Esta configuración proporciona al modelo información suficiente para capturar patrones diarios completos y ha sido utilizada de forma consistente en todas las arquitecturas evaluadas, a excepción de MICN.

Para el escenario de predicción a 48 horas ( $L_{out}^{48h} = 192$  pasos), mantener la misma relación implica utilizar una ventana de entrada de 384 pasos ( $L_{in}^{48h} = 384$ ). Sin embargo, durante las pruebas preliminares realizadas con la implementación de MICN de PyPOTS, esta configuración ha producido errores internos asociados a las operaciones convolucionales del modelo. Concretamente, tras las etapas de submuestreo y transformación multiescala, algunas capas convolucionales internas han intentado aplicar filtros de tamaño superior a la longitud efectiva de la secuencia procesada, colapsando la red.

Cabe destacar que el problema no se ha observado en configuraciones con  $L_{in} = 192$ , tanto para horizontes de 96 como de 192 pasos, lo que sugiere una limitación práctica de la implementación utilizada para determinadas combinaciones de ventana de entrada y horizonte de predicción, más que una limitación inherente a la arquitectura MICN.

Por este motivo, para el escenario de predicción a 48 horas se opta por adoptar una configuración con ventana de entrada igual al horizonte de predicción ( $L_{in}^{48h} = L_{out}^{48h} = 192$ ) para garantizar estabilidad durante el entrenamiento.

## 4.2 Análisis del Entrenamiento y Validación

Antes de someter las arquitecturas al conjunto de *test*, resulta imprescindible auditar la fase de entrenamiento y el proceso de optimización de hiperparámetros (HPO). El objetivo de esta fase metodológica es identificar la configuración estructural que maximice la capacidad de generalización de cada modelo, utilizando el MAE sobre el conjunto de validación como métrica objetivo.

La Tabla 4.1 consolida las métricas de error alcanzadas en validación por las configuraciones óptimas que resultan seleccionadas para la evaluación final.

Tabla 4.1: MAE alcanzado en el conjunto de validación tras la búsqueda de hiperparámetros.

| Tarea / Horizonte            | Modelo      | MAE (Validación) |
|------------------------------|-------------|------------------|
| <b>Imputación</b>            | SAITS       | 0.04961          |
|                              | CSDI        | 3.84810          |
| <b>Predicción (24 Horas)</b> | MICN        | 0.03914          |
|                              | DLinear     | 0.06295          |
|                              | Transformer | 0.25028          |
| <b>Predicción (48 Horas)</b> | MICN        | 0.04536          |
|                              | DLinear     | 0.06060          |
|                              | Transformer | 0.29357          |

Para indagar en las discrepancias de rendimiento expuestas en la tabla anterior, resulta indispensable auditar el comportamiento dinámico de las redes durante la fase de aprendizaje. La telemetría extraída de TensorBoard permite analizar las curvas de convergencia y diagnosticar las limitaciones subyacentes de cada topología.

### 4.2.1 Análisis de convergencia en modelos de imputación

El rendimiento anómalo de CSDI requiere un análisis visual detallado de sus curvas de aprendizaje.

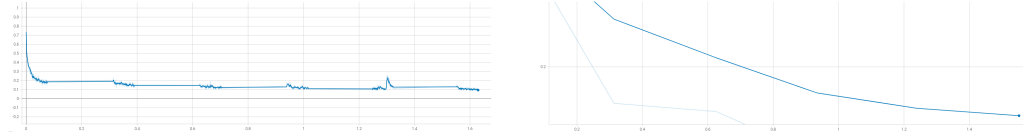


Figura 4.1: Curvas de convergencia para el modelo CSDI. Izquierda: evolución de la función de pérdida en entrenamiento. Derecha: función de pérdida en validación.

Al observar la Figura 4.1, la curva de entrenamiento muestra un descenso precipitado inicial seguido de un estancamiento prolongado. En CSDI, la función de pérdida mostrada en la gráfica no representa el MAE directo de la serie temporal, sino el error de estimación del ruido residual ( $\epsilon$ -matching) inyectado durante la cadena de Márkov. El modelo logra aprender a eliminar ruido gaussiano genérico con relativa rapidez, pero es incapaz de reconstruir la distribución real del tráfico urbano.

Como se documenta en la literatura especializada sobre difusión generativa [7], estas arquitecturas resultan extremadamente demandantes en términos de volumen de datos y ciclos de ajuste. La dificultad que enfrenta el modelo probabilístico para mapear el espacio latente multivariante con precisión no se atribuye exclusivamente a la limitación de la búsqueda paramétrica, sino también al establecimiento de un máximo de 20 épocas y a la insuficiencia de la longitud de los datos de entrenamiento para este tipo de arquitecturas.

#### 4.2.2 Análisis de convergencia en modelos de predicción

En el dominio de *forecasting*, los resultados de la Tabla 4.1 revelan una marcada superioridad de las arquitecturas MICN y DLinear frente al modelo basado en autoatención. Para justificar matemáticamente esta divergencia, resulta de gran utilidad contrastar de forma directa sus curvas de convergencia durante la fase de entrenamiento, ilustradas en la Figura 4.2.

El registro visual proporciona una confirmación empírica rotunda de la eficiencia computacional de las redes convolucionales frente a los mecanismos de atención puros en series temporales continuas. Al analizar la telemetría de MICN, se observa un desplome casi vertical del error durante los primeros compases del entrenamiento. La función de pérdida desciende rápidamente y se estabiliza de inmediato en valores próximos a cero: las convoluciones locales para alta frecuencia y las

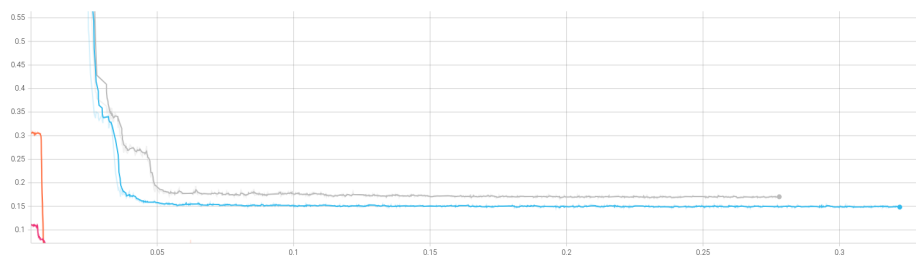


Figura 4.2: Comparativa de la función de pérdida en entrenamiento entre MICN (naranja y rosado) y Transformer (azul y gris) para ambos horizontes predictivos.

isométricas para tendencias globales extraen exitosamente las dinámicas del tráfico urbano.

En contraste, las trazas correspondientes a las ejecuciones del Transformer muestran una dinámica de aprendizaje mucho más ineficiente. El modelo inicia su ajuste con magnitudes de error significativamente mayores, y, aunque experimenta un descenso inicial pronunciado, esta caída deriva en un estancamiento prolongado que la red neuronal es incapaz de perforar (rango aproximado de 0.15 - 0.20). La causa estructural de este comportamiento se encuentra documentada en la literatura reciente. Como argumentan Zeng et al. [22], el mecanismo de autoatención puro es inherentemente invariante a las permutaciones. Aunque esta limitación teórica se mitiga mediante codificaciones posicionales (*positional encodings*), en el dominio de las series temporales numéricas y continuas esta estrategia resulta subóptima. El modelo atencional tiende a priorizar el cálculo de similitudes semánticas punto a punto, lo que provoca una pérdida de información sobre el orden cronológico estricto y limita su capacidad para abstraer la macro-tendencia frente a enfoques puramente lineales o convolucionales.

### 4.3 Resultados en Tareas de Imputación

Una vez aisladas y validadas las mejores configuraciones arquitectónicas, se evalúan los modelos neuronales SAITS y CSDI, así como los estimadores estadísticos de referencia (Media Global y LOCF) en el mismo conjunto de datos inexplorado de *test*. Los resultados obtenidos tras el proceso de inferencia se detallan en la siguiente tabla de métricas:

Tabla 4.2: Métricas de evaluación en el conjunto de test para tareas de imputación.

| Modelo / Algoritmo | MAE            | MSE            | RMSE           |
|--------------------|----------------|----------------|----------------|
| <b>SAITS</b>       | <b>0.05555</b> | <b>0.01189</b> | <b>0.10902</b> |
| Media Global       | 0.37965        | 0.26472        | 0.51451        |
| LOCF               | 0.53709        | 61.99049       | 7.87340        |
| CSDI               | 3.83389        | 100.37682      | 10.01882       |

El análisis de las métricas arroja conclusiones determinantes sobre la reconstrucción de la señal:

1. **El colapso del algoritmo de inercia (LOCF).** Aunque su Error Absoluto Medio (0.537) es deficiente pero acotado, su Error Cuadrático Medio se dispara de forma anómala (MSE de 61.99). Este fenómeno posee una explicación matemática fundamentada en la naturaleza del sensor: la ocupación de estacionamiento experimenta cambios de estado binarios o muy abruptos (por ejemplo, cuando un vehículo abandona la plaza). Si un sensor queda inactivo tras registrar un estado de alta ocupación, el algoritmo LOCF propaga ese estado de forma estática durante horas. Al elevar al cuadrado la diferencia acumulada frente a la realidad dinámica, la penalización escala masivamente, lo que demuestra empíricamente que la inercia temporal no es una heurística válida para entornos de movilidad volátiles.
2. **La superioridad determinista de SAITS.** La arquitectura SAITS domina la comparativa de forma absoluta: obtiene un MAE de 0.055 que ratifica la solidez de lo aprendido en validación. Al impedir que el modelo observe la coordenada exacta que debe predecir mediante el enmascaramiento diagonal, la red se ve forzada a asimilar la estacionalidad periódica extraída de los pasos adyacentes. Su bajísimo RMSE (0.109) confirma la práctica ausencia de *outliers* en su reconstrucción.
3. **La divergencia de CSDI.** El rendimiento de la arquitectura CSDI en el conjunto inexplorado (MAE de 3.833) es prácticamente diez veces superior al de un promedio simple como la Media Global. Como se aborda en la Sección 4.1, las severas restricciones computacionales merman el potencial de la arquitectura probabilística generativa.

## 4.4 Resultados en Tareas de Predicción

Por otro lado, se evalúa la capacidad de proyección futura de las arquitecturas neuronales en dos horizontes: a corto plazo (24 horas) y a medio plazo (48 horas). Las métricas arrojadas por la evaluación en el conjunto de prueba se consolidan en la siguiente comparativa:

Tabla 4.3: Métricas de evaluación en el conjunto de test para tareas de predicción a 24 y 48 horas.

| Horizonte | Modelo      | MAE            | MSE            | RMSE           |
|-----------|-------------|----------------|----------------|----------------|
| 24 Horas  | MICN        | <b>0.03848</b> | 0.02181        | 0.14768        |
|           | DLinear     | 0.06503        | <b>0.02013</b> | <b>0.14188</b> |
|           | Transformer | 0.26443        | 0.15471        | 0.39334        |
| 48 Horas  | MICN        | <b>0.04639</b> | 0.02492        | 0.15787        |
|           | DLinear     | 0.07342        | <b>0.02136</b> | <b>0.14615</b> |
|           | Transformer | 0.30828        | 0.17860        | 0.42261        |

El comportamiento predictivo de las redes neuronales se desglosa a continuación:

1. **La confirmación empírica del fracaso atencional puro.** Los resultados validan concluyentemente la tesis expuesta en el marco teórico respecto a las limitaciones topológicas del Transformer [22]. La arquitectura Transformer se sitúa como la más ineficiente en ambos escenarios.
2. **La similitud de rendimiento entre la convolución isométrica y la descomposición lineal.** La comparativa entre MICN y DLinear aporta el análisis empírico más interesante de la experimentación. **MICN se consolida como el modelo más preciso en términos de error absoluto** para ambos horizontes temporales. Su doble convolución estructural modela con enorme agudeza la volatilidad urbana diaria. Sin embargo, **DLinear obtiene los mejores registros en la minimización del error cuadrático** (MSE de 0.020 y 0.021). Al proyectar matemáticamente tendencias mediante capas densas directas, DLinear asume un comportamiento conservador: evita predecir anomalías extremas, lo que le otorga gran inmunidad frente a las penalizaciones del MSE, a expensas de sacrificar precisión fina.
3. **La resiliencia al aumentar el horizonte temporal.** Escalar el horizonte predictivo revela la solidez de la aproximación paramétrica, cuyo MAE



apenas experimenta degradación de 0.065 a 0.073. Por su parte, la arquitectura MICN muestra una caída predecible puesto que procesa la mitad del contexto histórico. Aun así, alcanza el mejor MAE.

## 4.5 Evaluación Compuesta del Pipeline Predictivo

Como culminación de la fase de experimentación, se ejecuta el flujo algorítmico integrado que incluye los dos subdominios de la investigación. El propósito de este ensayo empírico consiste en cuantificar si las arquitecturas de predicción experimentan una mejora significativa en su rendimiento al recibir un contexto histórico reconstruido previamente por SAITS (sin valores faltantes), frente a procesar directamente la señal cruda con los vacíos naturales de los sensores.

Para garantizar la rigurosidad del análisis, se seleccionan cuatro escenarios de prueba estratégicos con diferentes combinaciones de modelos y horizontes temporales. Los resultados obtenidos de someter el mismo conjunto de prueba a las redes de predicción de forma aislada y, posteriormente, a través del *pipeline* de preprocesamiento con SAITS, se consolidan en la Tabla 4.4.

Tabla 4.4: Resultados del *pipeline* integrado. Comparativa de error MAE sobre datos originales frente a datos previamente imputados por SAITS.

| Pipeline (Imputador → Predictor) | Estado   | MAE            | MSE            | Var. (%) |
|----------------------------------|----------|----------------|----------------|----------|
| SAITS → MICN (24H)               | Crudo    | 0.03845        | 0.02177        |          |
|                                  | Imputado | <b>0.03843</b> | <b>0.02165</b> | -0.05%   |
| SAITS → DLinear (24H)            | Crudo    | 0.06503        | 0.02010        |          |
|                                  | Imputado | <b>0.06471</b> | <b>0.02006</b> | -0.49%   |
| SAITS → DLinear (48H)            | Crudo    | <b>0.07342</b> | <b>0.02136</b> |          |
|                                  | Imputado | 0.07378        | 0.02136        | +0.49%   |
| SAITS → Transformer (24H)        | Crudo    | 0.26451        | 0.15475        |          |
|                                  | Imputado | <b>0.26440</b> | <b>0.15473</b> | -0.04%   |

Tras analizar los resultados obtenidos y ponderar los costes computacionales asociados, se concluye lo siguiente:

1. **El SOTA es robusto por naturaleza.** El primer escenario evalúa la sinergia entre dos modelos con alta precisión como son SAITS y MICN. En un inicio, es posible creer que si MICN exhibe un rendimiento sobresaliente asimilando datos con ruido, una señal bien imputada debería acercar el error a un mínimo teórico. Sin embargo, los resultados demuestran que la mejora es virtualmente inexistente, con una reducción del 0.05% en el error. Un fenómeno idéntico se observa al purificar la entrada de DLinear a 24 horas.

Este hallazgo es crucial: ambos modelos de *forecasting* actúan como mecanismos de suavizado resilientes para los valores nulos. La arquitectura interna de estos modelos asimila las anomalías de los sensores con tal eficacia que anula la necesidad de aplicar técnicas generativas de imputación previas.

2. **La propagación de error por escalabilidad.** Al someter el flujo a un escenario de alta exigencia (SAITS acoplado a DLinear con ventana histórica de 384 pasos), la imputación degrada ligeramente el rendimiento. Se observa el problema clásico de la propagación del error en inferencias en cascada: la reconstrucción de SAITS es muy precisa, aunque introduce pequeñas desviaciones sobre los vacíos del sensor, que en una ventana tan dilatada hacen que la señal acumule un sesgo sintético notable. Consecuentemente, se confunde ligeramente a la capa lineal del modelo predictivo, lo cual resulta más perjudicial que omitir el dato faltante original.
3. **La refutación algorítmica del Transformer:** Dado el rendimiento deficiente del Transformer sobre la señal original, se ha buscado determinar si su colapso obedece a una escasa tolerancia a los valores ausentes o a una deficiencia topológica profunda. Al inyectar la señal reconstruida, se observa que la mejora en el error predictivo es estadísticamente nula: se ratifica la hipótesis previamente planteada para el Transformer.

## 4.6 Evaluación sobre el *dataset Out-Of-Time*

Una vez se ha probado la correcta ejecución de los modelos en el conjunto histórico, es importante probar que estas tengan capacidad para generalizar en un entorno de producción. Por consiguiente, todos los modelos seleccionados se someten a una fase de evaluación sobre un subconjunto (por limitaciones de GPU)

del dataset de datos adquirido en tiempo real durante el mes de mayo. Al alimentar las redes con datos de un mes en el que no han sido entrenadas, se evalúa su robustez frente a la deriva conceptual.

#### 4.6.1 Resultados en tareas de imputación

La evaluación de los modelos preentrenados de imputación sobre los datos en tiempo real muestra los resultados consolidados en la Tabla 4.5.

Tabla 4.5: Métricas de evaluación OOT para modelos de imputación.

| Modelo / Algoritmo | MAE            | MSE            | RMSE           |
|--------------------|----------------|----------------|----------------|
| <b>SAITS</b>       | <b>0.03905</b> | <b>0.00969</b> | <b>0.09845</b> |
| Media Global       | 0.33509        | 0.24708        | 0.49707        |
| LOCF               | 0.53705        | 62.08458       | 7.87938        |
| CSDI               | 3.73094        | 97.74305       | 9.88651        |

Tras analizar los resultados se pueden observar *insights* interesantes. Contraintuitivamente, el modelo SAITS no solo ha resistido el salto temporal, sino que **ha mejorado su rendimiento** respecto a la validación histórica, con una reducción de su MAE de 0.055 a 0.039. Este hito ratifica que la red no ha memorizado las series de entrenamiento, sino que las capas de atención multiescala han logrado interiorizar con éxito la física subyacente de la ciudad y las estacionalidades temporales.

Por otro lado, el estimador LOCF vuelve a obtener resultados desorbitados en las métricas de error: queda invalidado como mecanismo de imputación dada la volatilidad de los cambios de estado binarios en el estacionamiento.

#### 4.6.2 Resultados en tareas de predicción

Se realiza un análisis de la proyección futura a 24 y 48 horas sobre datos OOT. Los resultados obtenidos se exponen en la Tabla 4.6.

Tabla 4.6: Métricas de evaluación OOT para tareas de predicción a 24 y 48 horas.

| Horizonte | Modelo         | MAE            | MSE            | RMSE           |
|-----------|----------------|----------------|----------------|----------------|
| 24 Horas  | <b>MICN</b>    | <b>0.04850</b> | <b>0.02802</b> | <b>0.16738</b> |
|           | DLinear        | 0.09287        | 0.03170        | 0.17804        |
|           | Transformer    | 0.24577        | 0.15382        | 0.39220        |
| 48 Horas  | <b>DLinear</b> | <b>0.09991</b> | <b>0.03086</b> | <b>0.17568</b> |
|           | Transformer    | 0.27709        | 0.17291        | 0.41582        |
|           | MICN (Lin=192) | 0.26417        | 0.46625        | 0.68282        |

El contraste de esta métricas de producción frente a la validación histórica revela hallazgos críticos sobre la escalabilidad de las redes neuronales temporales:

1. **Degradación de rendimiento por *concept drift*.** Se observa una ligera merma en la precisión de todas las redes. MICN se mantiene con el menor error en el corto plazo, mientras que DLinear sufre una penalización ligeramente mayor, pero conserva la solidez del error cuadrático (0.031 MSE).
2. **Gran robustez estructural de DLinear.** En el escenario de predicción a 48 horas vista, la arquitectura DLinear demuestra una resiliencia excepcional reteniendo un MAE de 0.099 y un MSE de 0.030, métricas notablemente superiores a las del resto de modelos. Su capacidad estructural para proyectar tendencias matemáticas directas le permite aislarse del ruido inédito introducido por la deriva conceptual, actuando como una línea base conservadora pero altamente fiable en el medio plazo.
3. **Colapso de MICN a 48 horas.** Es de gran relevancia el desplome absoluto de MICN en la predicción a 48 horas: su MAE se multiplica exponencialmente de 0.045 a 0.264, y su MSE alcanza 0.466. Como se ha justificado en la sección 4.1, las limitaciones técnicas de la red han forzado a usar un contexto histórico de entrada idéntico a su horizonte de predicción ( $L_{in} = L_{out} = 192$ ), lo cual merma la capacidad de generalización por falta de perspectiva temporal. Se demuestra empíricamente que la profundidad de la ventana de entrada es un factor de supervivencia innegociable en arquitecturas complejas sometidas a *concept drift*.

# 5 Conclusiones y trabajos futuros

## 5.1 Consecución de objetivos

Esta sección es la sección espejo de las dos primeras del capítulo de objetivos, donde se planteaba el objetivo general y se elaboraban los específicos.

Es aquí donde hay que debatir qué se ha conseguido y qué no. Cuando algo no se ha conseguido, se ha de justificar, en términos de qué problemas se han encontrado y qué medidas se han tomado para mitigar esos problemas.

Y si has llegado hasta aquí, siempre es bueno pasarle el corrector ortográfico, que las erratas quedan fatal en la memoria final. Para eso, en Linux tenemos *aspell*, que se ejecuta de la siguiente manera desde la línea de *shell*:

```
aspell --lang=es_ES -c memoria.tex
```

## 5.2 Aplicación de lo aprendido

Aquí viene lo que has aprendido durante el Grado/Máster y que has aplicado en el TFG/TFM. Una buena idea es poner las asignaturas más relacionadas y comentar en un párrafo los conocimientos y habilidades puestos en práctica.

1. a
2. b

## 5.3 Lecciones aprendidas

Aquí viene lo que has aprendido en el Trabajo Fin de Grado/Máster.

1. Aquí viene uno.
2. Aquí viene otro.

## 5.4 Trabajos futuros

Ningún proyecto ni software se termina, así que aquí vienen ideas y funcionalidades que estaría bien tener implementadas en el futuro.

Es un apartado que sirve para dar ideas de cara a futuros TFGs/TFMs.

## 5.5 Incorporación de código en la memoria

Es bastante habitual que se reproduzcan fragmentos de código en la memoria de un TFG/TFM. Esto permite explicar detalladamente partes del desarrollo que se ha realizado que se consideren de especial interés. No obstante, tampoco es conveniente pasarse e incluir demasiado código en la memoria, puesto que se puede alargar mucho el documento. Un recurso muy habitual es subir todo el código a un repositorio de un servicio de control de versiones como GitHub o GitLab, y luego incluir en la memoria la URL que enlace a dicho repositorio.

Para incluir fragmentos de código en un documento  $\text{\LaTeX}$  se pueden combinar varias herramientas:

- El entorno `\begin{listing}[]...\end{listing}` permite crear un marco en el que situar el fragmento de código (parecido al generado cuando insertamos una tabla o una figura). Podemos insertar también una descripción (*caption*) y una etiqueta para referenciarlo luego en el texto.

- Dentro de este entorno, se puede utilizar el paquete `minted`<sup>1</sup>, que utiliza el paquete Python Pygments para resaltado de sintaxis (coloreando el código). Como se puede ver en el siguiente ejemplo, hay muchas opciones de configuración que permiten controlar cómo se va a mostrar el código (incluir números de línea, saltos de línea, tamaño y tipo de fuente, espaciado, código de colores para resaltado, etc.).

```

^^I^^I# A dictionary is built to define the data type contained
↪ by each column
^^I^^Idtype_scheme = {'budget': np.int64, 'genres': np.object,
↪ 'homepage': np.str, 'id': np.int64, 'keywords': np.object,
↪ 'original_language': np.str, 'original_title': np.str,
↪ 'overview': np.str, 'popularity': np.float64,
↪ 'production_companies': np.object, 'production_countries':
↪ np.object, 'release_date': np.object, 'revenue': np.int64,
↪ 'runtime': np.float64, 'spoken_languages': np.object,
↪ 'status': np.object, 'tagline': np.str, 'title': np.str,
↪ 'vote_average': np.float64, 'vote_count': np.int64}
^^I^^I
^^I^^I# When loading the data from the .csv file, we provide the
↪ scheme to be followed for data typing
^^I^^Idf1 = dd.read_csv('tmdb_5000_movies.csv',
↪ dtype=dtype_scheme)

```

Código 5.1: Lectura de un fichero \*.csv y tipado de datos.

Otra ventaja del entorno `listing` es que se puede generar automáticamente un índice (con entradas hiperenlazadas) de fragmentos de código, para incluirlo al comienzo del documento junto con los índices de figuras, tablas, etc.

### 5.5.1 Fuentes monoespaciadas

A veces se incluyen nombres de archivos, paquetes, etc. como texto monoespaciado, utilizando el comando `LATEX\texttt{}`. Sin embargo, esto puede generar un problema cuando las palabras en fuente monoespaciada alcanzan el final de una línea. En ese caso, el compilador rehusa muchas veces romper la palabra y deja la línea demasiado larga respecto al resto.

Para evitar esto, especialmente en párrafos más cortos de lo habitual (como en una lista no numerada), se puede utilizar el entorno:

<sup>1</sup>[https://es.overleaf.com/learn/latex/Code\\_Highlighting\\_with\\_minted](https://es.overleaf.com/learn/latex/Code_Highlighting_with_minted)

```
^^I\begin{sloppypar}
^^I^^I...
^^I\end{sloppypar}
```

Se muestra a continuación con un ejemplo real.

- Los valores contenidos en las columnas `genres`, `spoken_languages`, `production_companies` y `production_countries`, clasificados originalmente como `np.objects`, se corresponden en realidad con listas de objetos JSON que han sido almacenadas como cadenas de caracteres. A través de la función `get_values(obj, key)` definida específicamente para ello, se transformará dicha cadena de caracteres en una lista de diccionarios a través de la función `json.loads(obj)` y se devolverá una tupla que recopile los valores de los mismos para la clave indicada, un objeto de Python mucho más manejable de cara a realizar consultas sobre el *dataset*.

## 5.6 Contenido matemático y ecuaciones

El contenido matemático se puede escribir en línea utilizando los delimitadores `\(...\)`, o bien delimitado por dos símbolos de `$`. Actualmente, se prefiere la primera opción para favorecer la compatibilidad con otras herramientas como Pandoc, así como para aprovechar el código para escribir ecuaciones con motores de interpretación de contenido matemático en HTML.

Si queremos que el contenido matemático se muestra en un bloque aparte, con una notación que ocupe más espacio vertical (para sumatorios, integrales, fracciones, etc.), entonces debemos usar el llamado *display mode*, delimitado por `\[...\]`, o bien por dos símbolos `$$`. De nuevo, la primera opción es preferible.

Por ejemplo, para mostrar en la misma línea una ecuación de segundo grado, escribimos  $3x^2 + 2x + 1 = 0$ . Si preferimos el contenido en un bloque independiente:

$$F(x) = \int_b^a \frac{1}{3}x^3$$

$$\frac{1}{\sqrt{x}}$$



$$\begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$$

Ahora bien, en un documento en el que tenemos que hacer referencia a las ecuaciones dentro del texto de explicación, es conveniente otorgarle a cada bloque matemático un identificador numérico, similar al que usamos en figuras y tablas. Para ello, empleamos el bloque `\begin{equation}` contenidos... `\end{equation}`. En esta ocasión ya no tenemos que incluir los delimitadores de contenido matemático en *display mode*.

$$\frac{1}{\sqrt{x}} \quad (5.1)$$

Si añadimos una etiqueta a la ecuación, entonces podemos referenciar la Ecuación 5.1 en el texto. Veamos más ejemplos con caracteres especiales.

$$\hat{f}(\omega) = \frac{1}{2\pi} \sum_{-\infty}^{\infty} f(x) e^{-i\omega x} dx, \quad (5.2)$$

$$\tilde{f}(\omega) = \frac{1}{2\pi} \int_{-\infty}^{\infty} f(x) e^{-i\omega x} dx, \quad (5.3)$$

$$\dot{\vec{\omega}} = \vec{r} \times \vec{I}. \quad (5.4)$$

### 5.6.1 Ecuaciones en múltiples líneas y elementos alineados

Algunos ejemplos que muestran alineamiento de ecuaciones en diferentes líneas con el entorno `\begin{align*}` ... `\end{align*}`.

|                   |                     |             |
|-------------------|---------------------|-------------|
| $x = y$           | $w = z$             | $a = b + c$ |
| $2x = -y$         | $3w = \frac{1}{2}z$ | $a = b$     |
| $-4 + 5x = 2 + y$ | $w + 2 = -1 + w$    | $ab = cb$   |

**Nota:** las ecuaciones anteriores en varias líneas tienen un modo de escritura matemática definido para cada línea, no globalmente (a nivel de ecuación).

$$\begin{aligned}f(x) &= x^2 \\g(x) &= \frac{1}{x} \\F(x) &= \int_b^a \frac{1}{3}x^3\end{aligned}$$

### 5.6.2 Teoremas y conjuntos

**Teorema 1** Para todo número entero no negativo  $n$ , tenemos:

$$(1+x)^n = \sum_{i=0}^n \binom{n}{i} x^i.$$

La expansión en series de Taylor de la función  $e^x$  viene dada por:

$$e^x = 1 + x + \frac{x^2}{2} + \frac{x^3}{6} + \dots = \sum_{n \geq 0} \frac{x^n}{n!}. \quad (5.5)$$

$$\forall x \in X, \quad \exists y \leq \epsilon,$$

$$\frac{n!}{k!(n-k)!} = \binom{n}{k}.$$

**Teorema 2** Para cualesquiera conjuntos  $A$ ,  $B$  y  $C$ , se cumple que:

$$(A \cup B) - (C - A) = A \cup (B - C).$$

# A Contenido adicional

## A.1 Dimensiones de página

A continuación se incluye un resumen de las dimensiones calculadas para el diseño de maquetación de las páginas de la memoria.

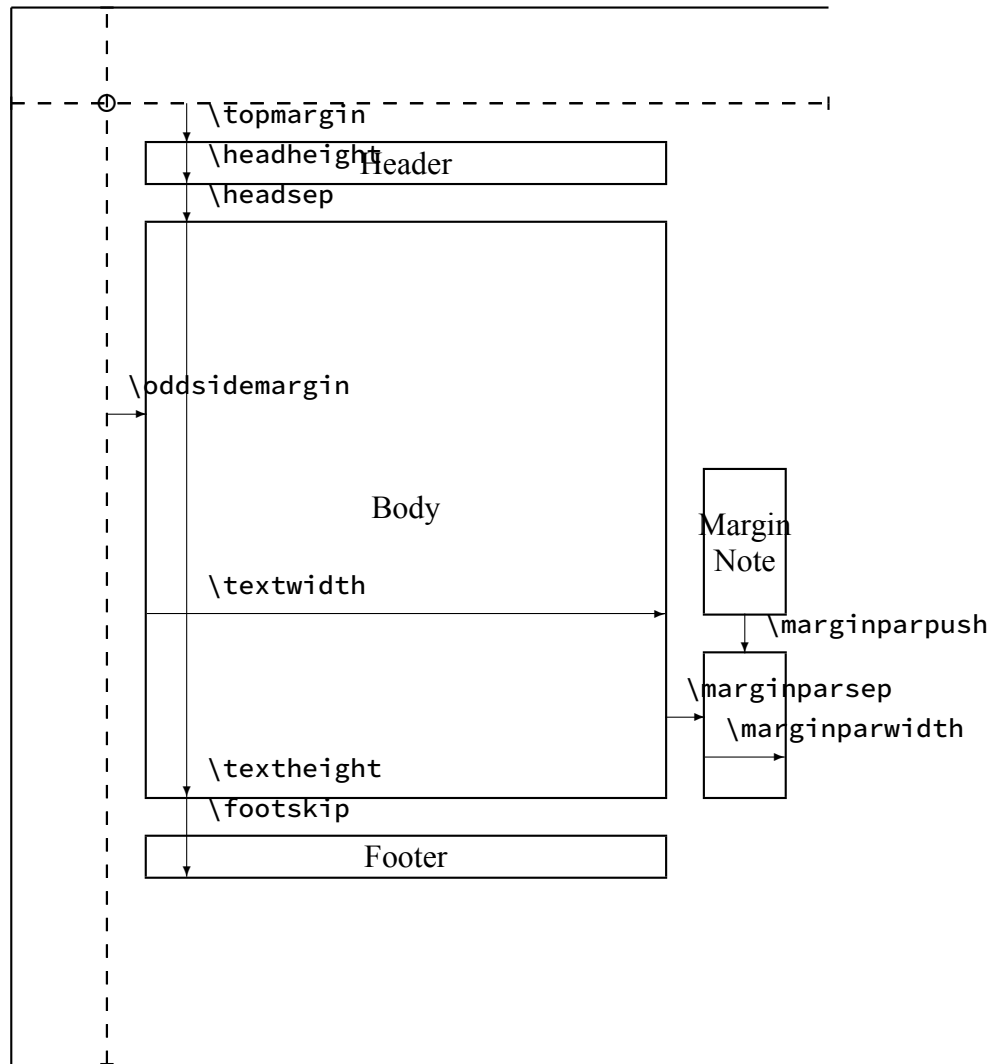
Actual page layout values.

|                                          |                                         |
|------------------------------------------|-----------------------------------------|
| <code>\paperheight = 296.99655mm</code>  | <code>\paperwidth = 209.99756mm</code>  |
| <code>\hoffset = 0mm</code>              | <code>\voffset = 0mm</code>             |
| <code>\evensidemargin = 14.1124mm</code> | <code>\oddsidemargin = 14.1124mm</code> |
| <code>\topmargin = -10.2513mm</code>     | <code>\headheight = 5.97475mm</code>    |
| <code>\headsep = 7.64415mm</code>        | <code>\textheight = 220.09608mm</code>  |
| <code>\textwidth = 135.97327mm</code>    | <code>\footskip = 17.83636mm</code>     |
| <code>\marginparsep = 4.51273mm</code>   | <code>\marginparpush = 1.40582mm</code> |
| <code>\columnsep = 3.51456mm</code>      | <code>\columnseprule = 0mm</code>       |
| <code>lem = 4.21747mm</code>             | <code>lex = 1.88632mm</code>            |

`\marginparwidth: 23.0081mm`

`\marginparwidth: 65.4652pt`

The circle is at 1 inch from the top and left of the page. Dashed lines represent (`\hoffset + 1 inch`) and (`\voffset + 1 inch`) from the top and left of the page.



Lorem ipsum dolor sit amet, consectetur adipiscing elit. Etiam lobortis facilisis sem. Nullam nec mi et neque pharetra sollicitudin. Praesent imperdiet mi nec ante. Donec ullamcorper, felis non sodales commodo, lectus velit ultrices augue, a dignissim nibh lectus placerat pede. Vivamus nunc nunc, molestie ut, ultricies vel, semper in, velit. Ut porttitor. Praesent in sapien. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Duis fringilla tristique neque. Sed interdum libero ut metus. Pellentesque placerat. Nam rutrum augue a leo. Morbi sed elit sit amet ante lobortis sollicitudin. Praesent blandit blandit mauris. Praesent lectus tellus, aliquet aliquam, luctus a, egestas a, turpis. Mauris lacinia lorem sit amet ipsum. Nunc quis urna dictum turpis accumsan semper. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Etiam lobortis facilisis sem. Nullam nec mi et neque pharetra sollicitudin. Praesent imperdiet mi nec ante. Donec ullamcorper, felis non sodales

commodo, lectus velit ultrices augue, a dignissim nibh lectus placerat pede. Vivamus nunc nunc, molestie ut, ultricies vel, semper in, velit. Ut porttitor. Praesent in sapien. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Duis fringilla tristique neque. Sed interdum libero ut metus. Pellentesque placerat. Nam rutrum augue a leo. Morbi sed elit sit amet ante lobortis sollicitudin. Praesent blandit blandit mauris. Praesent lectus tellus, aliquet aliquam, luctus a, egestas a, turpis. Mauris lacinia lorem sit amet ipsum. Nunc quis urna dictum turpis accumsan semper. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Etiam lobortis facilisis sem. Nullam nec mi et neque pharetra sollicitudin. Praesent imperdiet mi nec ante. Donec ullamcorper, felis non sodales commodo, lectus velit ultrices augue, a dignissim nibh lectus placerat pede. Vivamus nunc nunc, molestie ut, ultricies vel, semper in, velit. Ut porttitor. Praesent in sapien. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Duis fringilla tristique neque. Sed interdum libero ut metus. Pellentesque placerat. Nam rutrum augue a leo. Morbi sed elit sit amet ante lobortis sollicitudin. Praesent blandit blandit mauris. Praesent lectus tellus, aliquet aliquam, luctus a, egestas a, turpis. Mauris lacinia lorem sit amet ipsum. Nunc quis urna dictum turpis accumsan semper. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Etiam lobortis facilisis sem. Nullam nec mi et neque pharetra sollicitudin. Praesent imperdiet mi nec ante. Donec ullamcorper, felis non sodales commodo, lectus velit ultrices augue, a dignissim nibh lectus placerat pede. Vivamus nunc nunc, molestie ut, ultricies vel, semper in, velit. Ut porttitor. Praesent in sapien. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Duis fringilla tristique neque. Sed interdum libero ut metus. Pellentesque placerat. Nam rutrum augue a leo. Morbi sed elit sit amet ante lobortis sollicitudin. Praesent blandit blandit mauris. Praesent lectus tellus, aliquet aliquam, luctus a, egestas a, turpis. Mauris lacinia lorem sit amet ipsum. Nunc quis urna dictum turpis accumsan semper.



# Referencias

- [1] Amir H Alavi et al. «Internet of Things-enabled smart cities: State-of-the-art and future trends». En: *Measurement* 129 (2018), págs. 589-606.
- [2] J.J. Allaire et al. *Quarto*. Posit Software. 2022.  
URL: <https://github.com/quarto-dev/quarto-cli>  
(visitado 20-05-2024).
- [3] Karl J. Åström y Björn Wittenmark. *Computer-Controlled Systems: Theory and Design*. 3rd. Upper Saddle River, NJ, USA: Prentice Hall, 1997.
- [4] Wenjie Du, David Cote y Tianchun Liu. «PyPOTS: a Python toolbox for data mining on partially observed time series». En: *arXiv preprint arXiv:2305.10370* (2023).
- [5] Wenjie Du, David Cote y Yan Liu. «SAITS: Self-Attention-based Imputation for Time Series». En: *Expert Systems with Applications* 219 (2023), pág. 119619. DOI: [10.1016/j.eswa.2023.119619](https://doi.org/10.1016/j.eswa.2023.119619).
- [6] Mohammed Ali Al-Garadi et al. «A survey of machine and deep learning methods for internet of things (IoT) security». En: *IEEE Communications Surveys & Tutorials* 22.3 (2020), págs. 1646-1685.
- [7] Jonathan Ho, Ajay Jain y Pieter Abbeel. «Denoising diffusion probabilistic models». En: *Advances in Neural Information Processing Systems*. Vol. 33. Curran Associates, Inc., 2020, págs. 6840-6851.
- [8] Xuan Liang et al. «Assessing Beijing's PM2.5 pollution: severity, weather impact, APEC and winter heating». En: *Proceedings of the Royal Society A: Mathematical, Physical and Engineering Sciences* 471.2182 (2015), pág. 20150257.
- [9] Trista Lin, Hervé Rivano y Frédéric Le Mouel. «A survey of smart parking solutions». En: *IEEE Transactions on Intelligent Transportation Systems* 18.12 (2017), págs. 3229-3253.
- [10] Roderick JA Little y Donald B Rubin. *Statistical analysis with missing data*. Vol. 793. John Wiley & Sons, 2019.

- [11] Marek Miśkowicz. *Event-Based Control and Signal Processing*. Boca Raton, FL, USA: CRC Press, 2015.
- [12] Steffen Moritz et al. «Algorithm for time series imputation-and imputeTS package». En: *arXiv preprint arXiv:1510.03924* (2015).
- [13] Mark Raasveldt y Hannes Mühleisen. «DuckDB: an Embeddable Analytical Database». En: *Proceedings of the 2019 International Conference on Management of Data*. ACM. 2019, págs. 1981-1984.
- [14] Donald B. Rubin. «Inference and missing data». En: *Biometrika* 63.3 (1976), págs. 581-592.
- [15] Donald C Shoup. «Cruising for parking». En: *Transport Policy* 13.6 (2006), págs. 479-486.
- [16] Ikram Silva et al. «Predicting in-hospital mortality of ICU patients: The PhysioNet/Computing in Cardiology Challenge 2012». En: *2012 Computing in Cardiology*. IEEE. 2012, págs. 245-248.
- [17] Yusuke Tashiro et al. «CSDI: Conditional Score-based Diffusion Models for Probabilistic Time Series Imputation». En: *Advances in Neural Information Processing Systems*. Vol. 34. Curran Associates, Inc., 2021, págs. 24804-24816.
- [18] Guido Van Rossum y Fred L Drake Jr. *Python tutorial*. Centrum voor Wiskunde en Informatica Amsterdam, The Netherlands, 1995.
- [19] Ashish Vaswani et al. «Attention is all you need». En: *Advances in Neural Information Processing Systems*. Vol. 30. Curran Associates, Inc., 2017, págs. 5998-6008.
- [20] Huiqiang Wang et al. «MICN: Multi-scale Local and Global Context Modeling for Long-term Series Forecasting». En: *The Eleventh International Conference on Learning Representations (ICLR)*. 2023. URL: <https://openreview.net/forum?id=zt53IDUR1U>.
- [21] Haixu Wu et al. «Autoformer: Decomposition Transformers with Auto-Correlation for Long-Term Series Forecasting». En: *Advances in Neural Information Processing Systems*. Vol. 34. Curran Associates, Inc., 2021, págs. 22419-22431.
- [22] Ailing Zeng et al. «Are Transformers Effective for Time Series Forecasting?» En: *Proceedings of the Thirty-Seventh AAAI Conference on Artificial Intelligence*. Vol. 37. 9. AAAI Press, 2023, págs. 11121-11128.



- [23] Haoyi Zhou et al. «Informer: Beyond Efficient Transformer for Long Sequence Time-Series Forecasting». En: *Proceedings of the Thirty-Fifth AAAI Conference on Artificial Intelligence*. Vol. 35. 12. AAAI Press, 2021, págs. 11106-11115.